ProcessTensors.jl: An ITensor-native framework for non-Markovian open quantum dynamics

Gauthameshwar S. [1,2,⋆], **Aaron Sander** [3], **John F. Kam** [4,5], **Dario Poletti** [1,2,6], and **Kavan Modi** [1,2,†]

1 Singapore University of Technology and Design, Singapore
2 Centre for Quantum Technologies, Singapore
3 Technical University of Munich, Munich, Germany
4 Monash University, Melbourne, Australia
5 Agency for Science, Technology and Research (A*STAR), Singapore
6 MajuLab, CNRS-UNS-NUS-NTU International Joint Research Unit, Singapore

⋆ gauthameshwar_s@mymail.sutd.edu.sg , † kavan_modi@sutd.edu.sg

Abstract

We present ProcessTensors.jl, a Julia package for learning, constructing, and using process tensors with ITensor. The paper introduces process tensors from the ground up and connects the equations to tensor diagrams, tagged ITensor indices, and working code. Through guided examples, users can build process tensors, apply different sequences of quantum instruments, use testers with ancillary systems, and inspect memory across the temporal bonds. We implement exact and compressed ACE constructions and benchmark them against established software. We also review the current process-tensor software landscape and explain how the package can be extended with further PT-MPO construction methods, including ideas based on TEMPO, TEDOPA, and periodic process tensors. Our goal is to make process tensors approachable for new users while providing a clear foundation for future development of this package.

Received Date
Accepted Date
Published Date

Markers indicate whether a subsection is primarily theoretical (📓), focuses on Julia walkthroughs (♣), or both (📓♣).

Contents

1 Introduction

In principle, describing an open quantum system is easy: write down a giant wavefunction for the entire universe, place it in an even more gigantic Hilbert space, and evolve it according to the Schrödinger equation. Somewhere inside that wavefunction lies everything one could wish to know about the system of interest. In practice, of course, we cannot keep track of every surrounding particle, lattice vibration, stray electromagnetic field, or experimental imperfection and calculate their joint correlated dynamics. We therefore do what physicists have

always done when the universe becomes inconveniently large: draw a boundary. The degrees of freedom we wish to study are called the *system*; everything beyond this boundary is called the *environment* or *bath*. Open quantum-system theory asks how the bath affects the system without requiring us to simulate the bath in all its microscopic detail.

The most familiar answers we would see in this regard are those that rely on weak-coupling, short-memory, or related approximations on the baths that lead to time-local master equations. In the Markovian limit, the system does not need a record of its earlier history in order to determine what happens next. Markovian master equations provide an extremely successful workhorse for open quantum dynamics, with the Gorini–Kossakowski–Sudarshan–Lindblad form supplying the standard generator of quantum dynamical semigroups [1, 2]. Nevertheless, weak coupling and a clean separation of timescales are not available in every problem. Structured reservoirs, strong system–bath coupling, slow environmental modes, and delayed feedback can all preserve information about earlier dynamics [3,4]. In these regimes, memory is part of the physics rather than a correction that can safely be ignored.

The difficulty is not merely that the environment remembers, but that the system may give us no indication of *what* it remembers if we trace out the bath. To see this, consider two joint system–environment states at some time t_1,

$$\rho_{SE}^{(1)}(t_1) \qquad \text{and} \qquad \rho_{SE}^{(2)}(t_1), \tag{1}$$

which reduce to exactly the same system state,

$$\mathrm{Tr}_E\Big[\rho_{SE}^{(1)}(t_1)\Big] = \mathrm{Tr}_E\Big[\rho_{SE}^{(2)}(t_1)\Big] = \rho_S(t_1). \tag{2}$$

As far as the system density matrix is concerned, these two situations are indistinguishable. The environment, however, may disagree. It can retain different states or different correlations with the system in the two cases. Consequently, evolving both joint states with the same system–environment unitary need not produce the same reduced state at a later time:

$$\mathrm{Tr}_E\Big[U_{2:1}\rho_{SE}^{(1)}(t_1)U_{2:1}^\dagger\Big] \neq \mathrm{Tr}_E\Big[U_{2:1}\rho_{SE}^{(2)}(t_1)U_{2:1}^\dagger\Big]. \tag{3}$$

Thus, even perfect knowledge of the present system state need not be sufficient to predict its future. The information distinguishing the two experiments has simply moved somewhere beyond the boundary we drew earlier.

The limitation becomes sharper when we intervene our system along the way. Imagine performing two experiments with different histories and, at an intermediate time t_1, discarding the system state and preparing the same fresh state P in both. This operation—often called a *causal break*—removes any information that could pass directly from the past to the future through the system. Immediately afterward, both experiments therefore contain the same reduced state,

$$\rho_S^{(1)}(t_1^+) = \rho_S^{(2)}(t_1^+) = P. \tag{4}$$

Nevertheless, the intervention acts on a system that may already be correlated with its environment. A measurement outcome, for example, can condition the environment into a corresponding state, while repreparing P makes the system look identical in every run. If the later dynamics still depends on the earlier preparation or measurement outcome, then the memory must have travelled across the causal break through the environment. Such memory can remain hidden in an uninterrupted reduced trajectory, and can even occur when familiar two-time criteria describe the unperturbed evolution as Markovian [5].

This leaves a state-to-state description with an awkward question. What map should take P at t_1 to the state at t_2, if the same P can lead to different futures depending on what happened before it was prepared? There is no contradiction in quantum mechanics here; rather, we have

asked the reduced state to remember information that it no longer contains. The missing input is the history of interventions—the preparations, controls, and measurements through which the system and its environment arrived at the present experiment. A complete description of the quantum process must therefore accept an entire ordered sequence of operations, rather than only a density matrix at one particular time.

The process tensor provides precisely such a description [6–8]. For a sequence of interventions $(\mathcal{A}_0, \mathcal{A}_1, \ldots, \mathcal{A}_{n-1})$, it returns the final system state,

$$\rho_S(t_n) = \mathcal{T}_{n:0}\big[\mathcal{A}_{n-1}, \ldots, \mathcal{A}_0\big], \tag{5}$$

or, when the interventions contain measurements, the corresponding outcome probabilities and conditional states. A reduced trajectory tells us what happened under one prescribed history; a process tensor tells us what could have happened had we intervened differently along the way. It does not reveal every microscopic bath degree of freedom, but retains their complete influence on experiments performed through the system (the mathematical construction behind this statement—including its Choi representation and temporal input and output spaces—will be developed carefully in Sec. 3. For the present discussion, its operational meaning is enough). This makes causal breaks, multi-time correlations, instrument sequences, Markov order, and correlations across temporal partitions accessible within one description. In the Markovian limit, the process separates into independent transformations between successive times; otherwise, its temporal parts remain correlated, recording how information stored in the environment can return later. Memory thus becomes part of the object being manipulated rather than a feature inferred from a single trajectory.

The generality of a process tensor comes at an exponential cost: every additional intervention time introduces another pair of system indices, thus causing the exponential growth of the representation space of a process tensor, similar to how a many-body wavefunction space grows with the number of particles. Tensor networks make this manageable by representing the process as a matrix product operator in time, or PT-MPO [9]. The open legs provide the slots for system operations, while the internal bonds carry correlations between different parts of the process. When these temporal correlations are compressible, singular-value truncations reduce the representation to a controllable bond dimension. Different algorithms construct this PT-MPO by exploiting different structures of the environment: PT-TEMPO contracts the influence functional of Gaussian bosonic baths [9, 10], ACE combines and compresses the influence of independent bath modes [11], and time-translation-invariant approaches target long-time processes using repeating tensors [12]. The process tensor is therefore not made small by assumption; it is made useful whenever the environment stores its relevant memory in a compressible temporal space.

The principal open-source packages that construct reusable PT-MPOs are `OQuPy`, which implements PT-TEMPO for Gaussian bosonic environments [13], and the C++ `ACE` toolkit, which supports ACE, PT-TEMPO, and related contraction schemes for a broader class of independent bath modes [14]. Packages such as `QuantumDynamics.jl` and `MPSDynamics.jl` provide complementary influence-functional and explicit-environment tensor-network methods [15, 16]. Broader comparisons of these algorithms, software packages, and their detailed applicable regimes are already available in recent reviews [17, 18].

Then enters the hero of this paper, ProcessTensors.jl. BA BA BAA BAAMMMMMM. We are so easy to use. We also have a lot of documentation and examples. Someone just after learning basic Julia and reading this paper, can already install and use our software to fiddle with process tensors. We are the best. Just Kidding. The remaining part of the intro will be completed once we finalise the main contents of this scipost paper. Stay tuned!

2 From tensor diagrams to Liouville-space MPS and MPOs

The process tensor is a map acting on other maps. Before constructing such an object, it is useful to place density matrices and quantum operations on equal footing. Moving to the Liouville space does precisely this: density matrices become state-like vectors, while operations on density matrices become ordinary linear operators. This change of language is especially natural for tensor networks, because the same ideas can be read in three equivalent ways—as an equation, as a tensor diagram, and as the corresponding `ITensor` code. We develop these three descriptions together below. Our graphical conventions are adapted from the open-system tensor calculus of Ref. [19], with time and contractions drawn from right to left to match the convention used throughout this paper. In case the readers are not familiar with ITensors and their notations of sites and indices, they are encouraged to read the documentation of the ITensor library [cite ITensor docs, and also the original papers]. Alternatively, they can also go through the tutorials in the stable docs of our package [add the link to the stable docs].

2.1 A graphical dictionary for tensors

A tensor is represented by a shape with one open wire for every index. In our conventions, the direction of a wire records which dual space that leg belongs to: a leg pointing to the left is a ket index [Fig. 1a], whereas a leg pointing to the right is a bra index [Fig. 1b]. Joining two wires means summing over their shared index. For example, ordinary matrix multiplication,

$$(AB)_{ij} = A_{ik}B_{kj}, \tag{6}$$

is drawn by joining the right-hand wire of A to the left-hand wire of B [Fig. 1e]. Stacking two disconnected tensors vertically, with ket wires on the left and bra wires on the right, represents their tensor product [Fig. 1f]. Closing the two wires of an operator on the same index with a loop gives its trace [Fig. 1g],

$$\mathrm{Tr}(A) = A_{ii}. \tag{7}$$

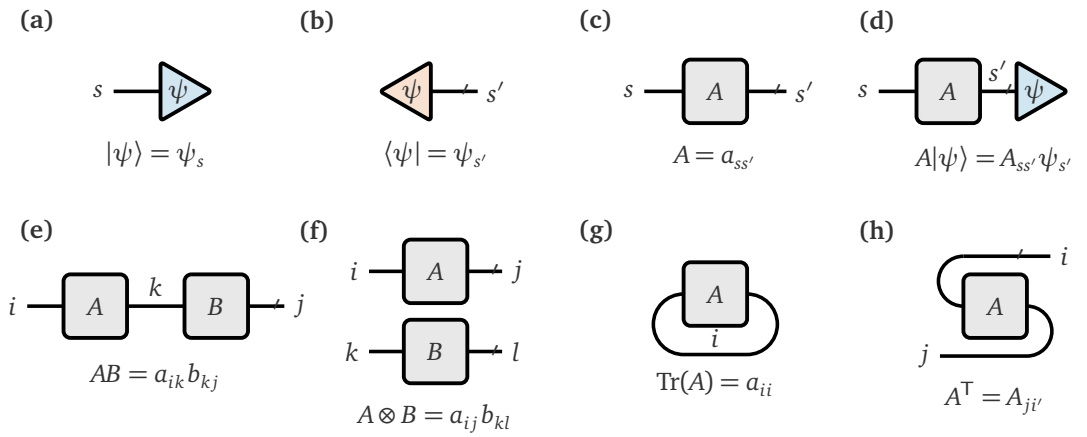

Figure 1: Graphical dictionary used throughout this work. (a) A ket $|\psi\rangle$ with unprimed index s, (b) A bra $\langle\psi|$ with primed index s', (c) An operator A with a ket wire s on the left and a bra wire s' on the right. (d) Contracting the primed wire of A with $|\psi\rangle$. (e) The matrix multiplication $AB = a_{ik}b_{kj}$. (f) The tensor product $A\otimes B = a_{ij}b_{kl}$, drawn by stacking A above B. (g) The trace $\mathrm{Tr}(A) = a_{ii}$. (h) The transpose $A^{\mathsf{T}} = A_{ji}$, obtained by bending the left and right wires to the opposite directions.

Equations (6) and (7), Fig. 1, and Listing 1 express the same basic rule: tensors contract when their indices match. Transposing deserves one warning when we implement it in ITensor

code. For an operator with indices s and s', `swapinds(A, s, prime(s))` exchanges those index identities and therefore implements A^{T}. An ITensor `prime` operation, by contrast, only changes the identity of an index; it does not transpose or conjugate the tensor. Likewise, `dag` conjugates the entries (and reverses index directions when applicable) but does not by itself exchange the two operator indices. This distinction becomes important when we use an unprimed index for the input and a primed copy for its output when constructing operator ITensors.

Julia

```julia
using ITensors

s = Index(2, "s")

psi = ITensor([1.0, 0.0], s) # (a) ket |psi> = psi_s

psidag = dag(prime(psi)) # (b) bra <psi| = psi_s'

A = random_itensor(s, prime(s)) # (c) operator A = a_ss'

Apsi = A * prime(psi) # (d) A|psi> = A_ss' psi_s'

i = Index(2, "i")
k = Index(2, "k")
j = Index(2, "j")
AB = random_itensor(i, k) * random_itensor(k, j) # (e) AB = a_ik b_kj

l = Index(2, "l")
AxB = random_itensor(i, j) * random_itensor(k, l) # (f) A ox B = a_ij
 ↪ b_kl  (no shared index)

TrA = A * delta(s, prime(s)) # (g) Tr(A) = a_ii

AT = swapinds(A, s, prime(s)) # (h) AT = a_ji
```

Listing 1: ITensor codes to realise tensor operations in Fig. 1(a)–(h).

2.2 From operator space to Liouville space

Let $\mathcal{H}$ be a d-dimensional Hilbert space of state vectors. Associated with it is the operator space

$$\mathcal{B}(\mathcal{H}) = \{A : \mathcal{H} \to \mathcal{H}\}, \tag{8}$$

containing all linear operators acting on $\mathcal{H}$. Density matrices, observables, Hamiltonians, and jump operators are therefore different inhabitants of the same space; the construction below is not restricted to Hermitian operators or physical states. The operator space becomes a d^2-dimensional Hilbert space when equipped with the Hilbert–Schmidt inner product

$$\langle\langle B|A\rangle\rangle = \mathrm{Tr}(B^\dagger A). \tag{9}$$

We refer to this operator Hilbert space as the *Liouville space*, $\mathcal{L}(\mathcal{H})$. The name reflects the historical description of dynamics directly in operator space, most notably through the quantum Liouville–von Neumann equation.

To use ordinary vector and tensor-network operations in this space, we choose a vector representation

$$A \in \mathcal{B}(\mathcal{H}) \longmapsto |A\rangle\!\rangle \in \mathcal{L}(\mathcal{H}), \tag{10}$$

called *vectorisation*. This does not turn A into a new physical state; it represents the same operator as a vector in Liouville space by rearranging the elements into a vector. In particular, density operators, observables, and non-Hermitian operators can all be vectorised in precisely the same way.

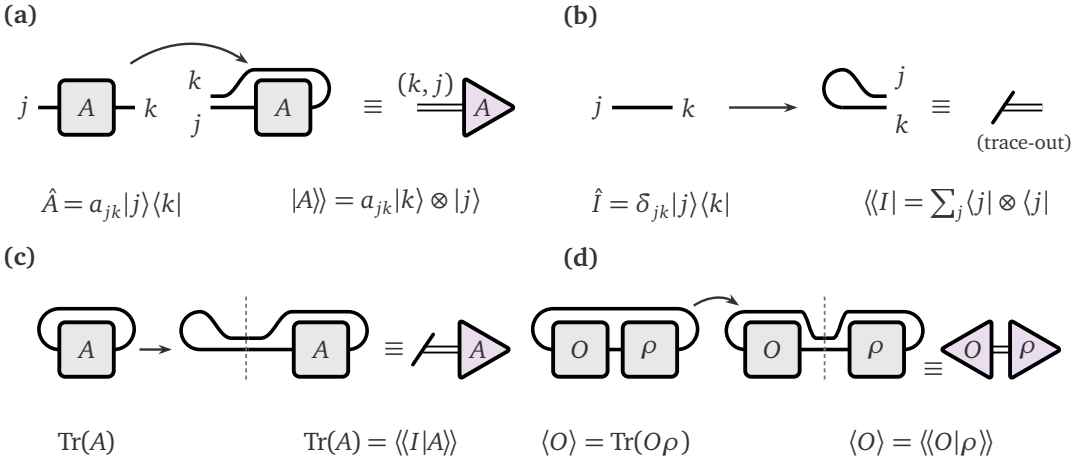

Figure 2: Vectorisation and Liouville-space closures. (a) An operator $\hat{A} = a_{jk}|j\rangle\langle k|$ is vectorised by bending the bra index k over the top, giving column-major order. The fused pair is a double Liouville wire. (b) The identity $\hat{I} = \delta_{jk}|j\rangle\langle k|$ is a straight wire. Bending the left index to the right yields the trace-out operation. (c) The Hilbert-space trace is the Liouville inner product $\mathrm{Tr}(A) = \langle\!\langle I|A\rangle\!\rangle$. (d) An expectation value is the same contraction with an observable, $\langle O\rangle = \langle\!\langle O|\rho\rangle\!\rangle$.

Different vectorisation conventions arrange the matrix entries differently. The package follows Julia's column-major ordering. For $a_{jk} = \langle j|A|k\rangle$, we define

$$|A\rangle\!\rangle \equiv \mathrm{vec}(A) = \sum_{j,k} a_{jk}|k\rangle \otimes |j\rangle, \tag{11}$$

where the first factor records the bra index k (slowest varying index) and the second the ket index j (fastest varying index), and the new double-wired index is a Liouville wire of the tuple (k, j). This is represented as a bent wire with two indices pointing in the same direction in Fig. 2a (note that if we bend the ket index k downwards, that would correspond to a row-major order because the tuple has j as the slowest varying index, and k as the fastest varying index, i.e. the order (j, k)). Equivalently, if we look at the matrix elements of A as a 2D array, the columns of A are stacked vertically in a column-major order:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \longmapsto \begin{pmatrix} a_{11} \\ a_{21} \\ a_{12} \\ a_{22} \end{pmatrix}. \tag{12}$$

Thus,

$$\mathrm{vec}\big(|j\rangle\langle k|\big) = |k\rangle \otimes |j\rangle. \tag{13}$$

An important Liouville vector used repeatedly below is the vectorised identity. Graphically, it is represented as a bent wire with two indices pointing in the same direction,

$$|I\rangle\rangle = \sum_j |j\rangle \otimes |j\rangle, \tag{14}$$

as also shown in panel (b) of Fig. 2. Using Eq. (9) and substituting B with the identity operator, we see in Fig. 2(c) that the corresponding trace operation in Liouville space is expressed as an inner product:

$$\mathrm{Tr}(A) = \langle\langle I|A\rangle\rangle. \tag{15}$$

The identity operator, I, is the maximally mixed state up to a normalisation $\rho_{\mathrm{mm}} = I/d$. Vectorising it (panel (b)) graphically yields the operation of closing a leg, or tracing out that subsystem, as illustrated in Fig. 2(b) and (c). More generally, an observable $O = O^\dagger$ closes the same Liouville leg and returns its expectation value, as depicted in Fig. 2(d):

$$\mathrm{Tr}(\rho) = \langle\langle I|\rho\rangle\rangle, \qquad \langle O\rangle = \mathrm{Tr}(O\rho) = \langle\langle O|\rho\rangle\rangle. \tag{16}$$

Leaving the leg unclosed, as in panel (a), instead retains the state-like object $|\rho\rangle\rangle$. These closure rules, visualised in the respective panels of Fig. 2, will reappear when process-tensor outputs are traced, measured, or left open.

Listing 2 realises the four panels of Fig. 2 with the package converters: an operator becomes a Liouville vector, the identity becomes the trace-out effect, and traces or expectation values are ordinary inner products.

Julia

```julia
using ProcessTensors

sites = siteinds("S=1/2", 1)
sites_L = liouv_sites(sites)

A = OpSum()
A += 1.0, "Sx", 1
A_L = to_liouville(MPO(A, sites); sites=sites_L) # (a) |A>>

Id = OpSum()
Id += 1.0, "Id", 1
Id_L = to_liouville(MPO(Id, sites); sites=sites_L) # (b) <<I|

TrA = inner(Id_L, A_L) # (c) Tr(A) = <<I|A>>

rho_L = to_liouville(to_dm(MPS(sites, ["Up"])); sites=sites_L)
O = OpSum()
O += 1.0, "Sz", 1
O_L = to_liouville(MPO(O, sites); sites=sites_L)
expect_O = inner(O_L, rho_L) # (d) <O> = <<O|rho>>
```

Listing 2: Vectorisation, the identity, and Liouville closures matching Fig. 2(a)–(d).

2.3 Left and right actions

Vectorisation becomes especially useful when we want to describe an operation such as $A\rho B$ as a linear action on the Liouville-space vector $|\rho\rangle\rangle$, because it brings the opeartions on the bra

and ket bases to the same space. This allows Hamiltonian evolution, dissipative jumps, and process-tensor contractions to be written as ordinary tensor-network operations or MPOs. The only small surprise is that multiplication from the right does not look exactly like multiplication from the left after vectorisation. The transpose appearing below is the bookkeeping rule that keeps the two descriptions consistent.

This viewpoint is closely related to the Choi–Jamiołkowski isomorphism [20, 21]. A linear map $\Phi : \mathcal{B}(\mathcal{H}) \to \mathcal{B}(\mathcal{H})$ can be represented in several equivalent ways. It may be described by its action on operators,

$$\rho \longmapsto \Phi(\rho),$$

by a Liouville-space superoperator S_Φ,

$$|\Phi(\rho)\rangle\!\rangle = \mathsf{S}_\Phi |\rho\rangle\!\rangle,$$

or by its Choi operator,

$$J(\Phi) = (\mathcal{I} \otimes \Phi)\big(|I\rangle\!\rangle\langle\!\langle I|\big).$$

The Liouville-space matrix and the Choi operator contain the same map, but organise its input and output indices differently. In this work, we use the Liouville-space representation because it is the form that can be applied directly to tensor-network states. The Choi representation is particularly useful when studying complete positivity and quantum channels; introductory discussions can be found in Refs. [22, 23]. Fidel: Check if this para is sound, and see if you need more elaboration/correction.

The connection between operator multiplication and Liouville-space action is most easily seen by sliding an operator around the same bend used to vectorise the identity in Fig. 2b. For the vectorised identity,

$$(I \otimes A)|I\rangle\!\rangle = (A^{\mathsf{T}} \otimes I)|I\rangle\!\rangle. \tag{17}$$

Graphically, A sits on the ket rail of the cup; after it slides through the bend it reappears transposed on the bra rail [Fig. 3a]. The transpose is not an additional physical operation; it records the change in index ordering induced by our column-major vectorisation convention. Applying this identity to a product gives Roth's lemma,

$$\mathrm{vec}(A\rho B) = (B^{\mathsf{T}} \otimes A)|\rho\rangle\!\rangle. \tag{18}$$

Figure 3b draws the same statement: A, ρ, and B on the ket rail are equivalent to B^{T} and A acting on the two legs of the bent ρ, which after connecting to Fig. 1f, gives the Liouville operator $B^{\mathsf{T}} \otimes A$ on $|\rho\rangle\!\rangle$. Thus the same transformation can be viewed in three equivalent ways,

$$A\rho B \quad \longleftrightarrow \quad \Phi_{A,B}(\rho) = A\rho B \quad \longleftrightarrow \quad \mathsf{S}_{A,B} = B^{\mathsf{T}} \otimes A.$$

The first expression is ordinary operator multiplication, the second is a linear map acting on operator space, and the third is its matrix representation in Liouville space.

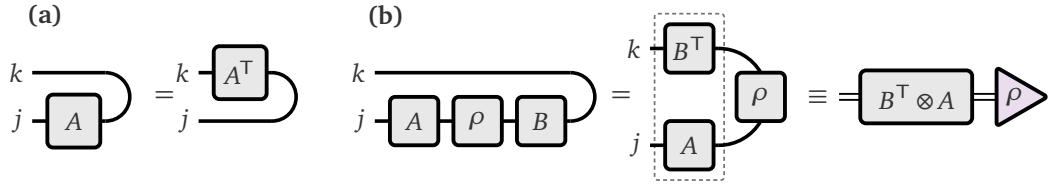

Figure 3: Wire sliding and Roth's lemma. (a) An operator that slides through the cup emerges transposed. (b) The same move on a product yields $\mathrm{vec}(A\rho B) = (B^{\mathsf{T}} \otimes A)|\rho\rangle\!\rangle$.

The two special cases used throughout the package are

$$A\rho \longleftrightarrow (I \otimes A)|\rho\rangle\rangle \equiv A_{\mathrm{L}}|\rho\rangle\rangle, \tag{19}$$

$$\rho B \longleftrightarrow (B^{\mathsf{T}} \otimes I)|\rho\rangle\rangle \equiv B_{\mathrm{R}}|\rho\rangle\rangle. \tag{20}$$

Setting $B = I$ in Eq. (18) leaves A on the ket rail and gives the left action A_{L}, likewise setting $A = I$ leaves B^{T} on the bra rail and gives the right action B_{R}. The suffixes L and R describe multiplication from the left and right of the density operator before vectorisation. Listing 3 realises the two special cases of Fig. 3b, together with the left-right jump that follows from the same identity.

Julia

```julia
using ProcessTensors

sites = siteinds("S=1/2", 1)
sites_L = liouv_sites(sites)
sL = only(sites_L)

A_L = op("Sx_L", sL)         # vec(Sx * rho)
B_R = op("Sz_R", sL)         # vec(rho * Sz)
A_jump = op("S-_Jump", sL)   # vec(S- * rho * S+)
```

Listing 3: Package notation for left, right, and left-right actions.

In this notation, `Sx_L` represents multiplication by S_x from the left, while `Sz_R` represents multiplication by S_z from the right. The `_Jump` suffix represents the corresponding left-right action, also known as the sandwich action in the context of Local Master Equation (LME) superoperators. Together, these local operators provide the elementary building blocks for the Liouville-space dynamics and process-tensor constructions developed below.

2.4 Vectorised many-body MPS and Liouvillian MPOs

A single quantum degree of freedom lives in one Hilbert space. A many-body system brings several of them together: if site j has a local Hilbert space $\mathcal{H}_j$ of dimension d_j, then the full state belongs to the joint Hilbert space

$$\mathcal{H} = \bigotimes_{j=1}^{N} \mathcal{H}_j, \qquad \dim \mathcal{H} = \prod_{j=1}^{N} d_j. \tag{21}$$

The price of adding sites is therefore immediate: the number of amplitudes grows exponentially. Tensor networks tame this growth by storing the local degrees of freedom in the form of a matrix-product state (MPS),

$$|\psi\rangle = \sum_{s_1,\ldots,s_N} A_1^{s_1} A_2^{s_2} \cdots A_N^{s_N} |s_1 \cdots s_N\rangle. \tag{22}$$

Neighbouring cores are joined by virtual bonds. Their dimensions control how much correlation and entanglement the MPS can represent across each spatial cut, and truncating them provides the compression that makes the calculation practical. Standard introductions to MPS and matrix-product operators (MPOs) can be found in Refs. [24, 25]; the ITensor implementation used here is described in Ref. [26].

A mixed many-body state is described by a density operator on this joint Hilbert space,

$$\rho = \sum_{\boldsymbol{s},\boldsymbol{s}'} \rho_{\boldsymbol{s},\boldsymbol{s}'} |s_1 \cdots s_N\rangle\langle s'_1 \cdots s'_N|, \tag{23}$$

where $\boldsymbol{s} = (s_1, \ldots, s_N)$. When converted to an MPO, each local core now has two physical legs: an unprimed ket index s_j and a primed bra index s'_j, along with the virtual bond indices connecting adjacent cores.

Vectorisation of this density operator can now be performed one site at a time. As shown in Fig. 4, bending each bra wire towards its ket wire produces the column-major pair (s'_j, s_j). A local combiner C_j then fuses this pair into the Liouville index

$$\alpha_j = (s'_j, s_j), \qquad C_j : \mathcal{H}_j^* \otimes \mathcal{H}_j \longrightarrow \mathcal{L}_j, \qquad \dim \mathcal{L}_j = d_j^2. \tag{24}$$

Applying these combiners across the chain turns the `MPO{Hilbert}` representation of ρ into an `MPS{Liouville}` representation of $|\rho\rangle\rangle$. The virtual bonds are untouched; only the two local physical legs are fused. Thus vectorisation changes how the same object is represented, without introducing a second physical state.

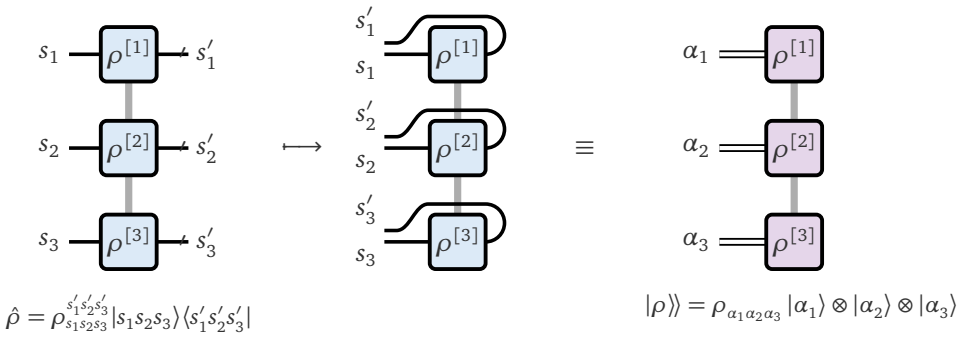

Figure 4: Local many-body vectorisation of a three-site density matrix. A Hilbert-space `MPO{Hilbert}` has an unprimed ket s_j and a primed bra s'_j on every site. Bending each bra over its ket gives the column-major pair (s'_j, s_j), which is fused into the Liouville index α_j of the resulting `MPS{Liouville}`.

In `ProcessTensors.jl`, `to_dm` constructs a Hilbert-space density MPO and `to_liouville` applies the local fusion. The resulting Liouville MPS can then be contracted with vectorised observables or the vectorised identity, exactly as in the single-site discussion.

Julia

```julia
using ProcessTensors

sites = siteinds("S=1/2", 3)
sites_L = liouv_sites(sites)

psi1 = MPS(sites, ["Up", "Up", "Up"])
psi2 = MPS(sites, ["Dn", "Dn", "Dn"])
rho = to_dm([psi1, psi2]; coeffs=[0.8, 0.2])
rho_L = to_liouville(rho; sites=sites_L)

Id = OpSum()
Id += 1.0, "Id", 1
O = OpSum()
```

```
O += 1.0, "Sz", 2

Id_L = to_liouville(MPO(Id, sites); sites=sites_L)
O_L = to_liouville(MPO(O, sites); sites=sites_L)
normalisation = inner(Id_L, rho_L)
expectation = inner(O_L, rho_L)
```

Listing 4: Three-site mixed state matching Fig. 4.

There is a point worth noting while defining the Liouville indices and using them to construct the corresponding Liouville-space object. The Liouville indices should be created once and reused. Calling `liouv_sites` independently produces indices with the same dimensions and tags but different identities (IDs), so ITensor will not contract them as the same legs. The same issue appears when one constructs a density matrix without inputting the `liouv_sites`. This would internally create a new set of indices from the Hilbert-space indices with a different index ID.

For a closed system, the density operator obeys the von Neumann equation

$$\frac{d\rho(t)}{dt} = -i[H,\rho(t)] = -i\big(H\rho(t) - \rho(t)H\big). \tag{25}$$

Using the left and right actions from Eqs. (19) and (20), this becomes one operator acting on a Liouville vector,

$$\frac{d}{dt}|\rho(t)\rangle\!\rangle = \mathcal{L}_H|\rho(t)\rangle\!\rangle, \qquad \mathcal{L}_H = -i\big(H_{\mathrm{L}} - H_{\mathrm{R}}\big). \tag{26}$$

For an `OpSum` $H = \sum_m h_m$, the package constructs this generator term by term: every operator string h_m is converted into its `_L` and `_R` actions before the Liouville MPO is built. It does not first construct the full Hilbert-space Hamiltonian MPO and vectorise that object. This keeps the construction local and preserves the structure already present in the `OpSum`.

Going beyond the unitary time evolution, the most common Markovian open-system generator is the Gorini–Kossakowski–Lindblad–Sudarshan (GKLS) master equation,

$$\frac{d\rho}{dt} = -i[H,\rho] + \sum_\mu \gamma_\mu \left(L_\mu \rho L_\mu^\dagger - \frac{1}{2} L_\mu^\dagger L_\mu \rho - \frac{1}{2} \rho L_\mu^\dagger L_\mu \right). \tag{27}$$

The dictionary developed above translates it directly into

$$\mathcal{L} = -i\big(H_{\mathrm{L}} - H_{\mathrm{R}}\big) + \sum_\mu \gamma_\mu \left[(L_\mu)_{\mathrm{Jump}} - \frac{1}{2}(L_\mu^\dagger L_\mu)_{\mathrm{L}} - \frac{1}{2}(L_\mu^\dagger L_\mu)_{\mathrm{R}} \right], \tag{28}$$

where $(L_\mu)_{\mathrm{Jump}} = L_\mu^* \otimes L_\mu$. The package suffixes `_Jump`, `_LdagL_L`, and `_LdagL_R` implement precisely these three dissipative terms.

Julia

```
H = OpSum()
H += 0.7, "Sx", 1
H += 0.2, "Sz", 1

L_H = liouvillian_mpo(H, sites_L)
L_open = liouvillian_mpo(
    H,
    sites_L;
```

```
    jump_ops=[(0.1, "S-", 1)],
)
```

Listing 5: Constructing Hamiltonian and dissipative Liouville MPOs.

We have arrived at a familiar place where we have described the generator of the Liouville-space dynamics in the form of a Liouville MPO. This can be readily used to perform open-quantum system dynamics, analogous to how we use the Hamiltonian MPO to perform unitary time evolution. Over a short interval Δt, the time-evolution operator is given by

$$|\rho(t+\Delta t)\rangle\!\rangle = e^{\Delta t \mathcal{L}} |\rho(t)\rangle\!\rangle. \tag{29}$$

Knowing $\mathcal{L}$, one can approximate the exponent of this MPO by using Suzuki-Trotter decomposition, as in TEBD cite TEBD literature, or one can variationally optimize the time-evolved state knowing the generator using TDVP cite TDVP literature to simulate open-quantum dynamics. Appendix B gives minimal examples where we can use the existing algorithms in `ITensorMPS.jl`, extend it to the Liouville space, and perform open-quantum dynamics.

Table 1: Hilbert-space expressions, Liouville-space representations, and their `ProcessTensors.jl` counterparts.

Hilbert-space object	Liouville-space object	Package representation
$\|\psi\rangle$	state vector	`MPS{Hilbert}`
ρ	$\|\rho\rangle\!\rangle$	`MPS{Liouville}`
density operator	ket–bra legs fused locally	`to_liouville`
$A\rho$	$A_{\mathrm{L}}\|\rho\rangle\!\rangle$	`"A_L"`
ρA	$A_{\mathrm{R}}\|\rho\rangle\!\rangle$	`"A_R"`
$A\rho B$	$(B^{\mathsf{T}} \otimes A)\|\rho\rangle\!\rangle$	left–right action
$A\rho A^\dagger$	$(A^* \otimes A)\|\rho\rangle\!\rangle$	`"A_Jump"`
$\mathrm{Tr}\,\rho$	$\langle\!\langle I\|\rho\rangle\!\rangle$	`inner(Id_L,rho_L)`
$\langle O\rangle$	$\langle\!\langle O\|\rho\rangle\!\rangle$	`inner(O_L,rho_L)`
$-i[H,\rho]$	$\mathcal{L}_H\|\rho\rangle\!\rangle$	`liouvillian_mpo`

3 Process tensor theory and construction

The previous section placed density operators, quantum operations, and many-body generators within a common Liouville-space tensor-network language. We now use that language to describe dynamics in which the system may be intervened upon at several times. In this section, we first show why a state-to-state quantum channel is insufficient for such experiments and introduce the process tensor as its multi-time generalisation. We then connect its operational and Choi representations to the temporal MPO stored by `ProcessTensors.jl`, identify the system and memory indices carried by its cores, and assemble the microscopic ingredients required for its construction. Finally, we construct the same `ProcessTensor` representation using the dense and ACE builders with some details on the compression methods used by `ACE()`.

3.1 From quantum channels to multi-time processes

Let us zoom out from the tensor-network details of Sec. 2 for a moment. There, density operators were vectorised, generators were written as many-body MPOs, and their action became

an ordinary tensor contraction. Beneath this machinery lies a simple physical requirement: a state evolving from one time to another must remain a valid quantum state. In particular, it must remain Hermitian and positive so that measurement probabilities are real and non-negative, and it must retain unit trace so that those probabilities add up to one. A quantum channel is the linear map that enforces these requirements. Writing

$$\rho' = \Phi[\rho], \qquad \Phi : \mathcal{B}(\mathcal{H}_{\text{in}}) \longrightarrow \mathcal{B}(\mathcal{H}_{\text{out}}), \tag{30}$$

the map Φ is required to be completely positive and trace preserving (CPTP),

$$(\Phi \otimes \mathcal{I}_A)[X] \geq 0 \qquad \text{for every } X \geq 0, \tag{31}$$

$$\text{Tr}(\Phi[\rho]) = \text{Tr}(\rho). \tag{32}$$

The word *complete* is important. Positivity must be preserved not only for an isolated system, but also when the map acts on one part of a larger state entangled with an otherwise untouched ancilla A. Thus, behind the formal acronym "CPTP" are the familiar probability rules of quantum mechanics, together with the demand that they remain consistent when the system is embedded in a larger experiment [8].

We have already encountered one such example, although we did not call it a channel at the time. In Sec. 2, Markovian open-system dynamics was generated by the Gorini–Kossakowski–Lindblad–Sudarshan (GKLS) equation

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = \mathcal{L}_t[\rho] = -\mathrm{i}[H(t), \rho] + \sum_\mu \gamma_\mu(t)\left(L_\mu \rho L_\mu^\dagger - \frac{1}{2}\left\{L_\mu^\dagger L_\mu, \rho\right\}\right), \qquad \gamma_\mu(t) \geq 0. \tag{33}$$

Strictly speaking, the Liouvillian $\mathcal{L}_t$ is the generator rather than the channel itself. The corresponding propagator

$$\Phi_{t:s} = \overleftarrow{\mathcal{T}} \exp\left[\int_s^t \mathrm{d}\tau\, \mathcal{L}_\tau\right], \qquad \rho(t) = \Phi_{t:s}[\rho(s)], \tag{34}$$

is CPTP; for a time-independent generator this reduces to $\Phi_{t:s} = \exp[(t-s)\mathcal{L}]$. The Liouvillian MPO introduced previously is therefore the tensor-network representation of the generator whose propagation carries one density operator to another without leaving the space of physical states.

A second, and more general, route to a quantum channel is to regard the apparent non-unitarity of the system as part of a larger unitary evolution. It is straightforward to see that a unitary transformation on a many-body denstiy operator is a CPTP map: they preserve the state norm by definition, $U\rho U^\dagger$ is Hermitian if ρ is Hermitian, and it keeps the same eigenvalues under the transformation, ensuring positivity. If the system begins uncorrelated with an environment in a fixed state ρ_E, then

$$\Phi_{t:s}[\rho_S] = \text{Tr}_E\left[U_{t:s}(\rho_S \otimes \rho_E)U_{t:s}^\dagger\right] = \sum_\mu K_\mu \rho_S K_\mu^\dagger, \tag{35}$$

where the Kraus operators obey

$$\sum_\mu K_\mu^\dagger K_\mu = I_S. \tag{36}$$

In finite dimensions, a linear map is CPTP if and only if it admits such a Kraus representation; equivalently, every quantum channel can be realised by adjoining an initially uncorrelated environment, evolving the joint state unitarily, and discarding the environment afterwards [8]. Unitary evolution and GKLS propagation are therefore not separate ideas, but familiar members of the same family of physical state-to-state maps.

All these channels answer essentially the same question: given the state of the system now, what will its state be later? For a memoryless evolution, this state-to-state picture may be applied successively,

$$\rho_S(t_{k+1}) = \Phi_{k+1:k}[\rho_S(t_k)]. \tag{37}$$

But must the map $\Phi_{k+1:k}$ be determined by the present reduced state alone? Suppose two experimental histories produce exactly the same $\rho_S(t_k)$, while leaving behind different correlations between the system and its environment. An operation performed at an earlier time may have disturbed the environment, conditioned it on a measurement outcome, or stored information in degrees of freedom that are no longer visible in $\rho_S(t_k)$. The same current system state can then have different futures. In that situation, the question "what state comes next?" has omitted part of the experimental input: "how did we reach the present state?".

The natural remedy is to promote the history of interventions to an input of the dynamical description. Let $\mathcal{A}_k$ denote the operation applied to the system at time t_k, while the joint system–environment dynamics between consecutive times is described by $\mathcal{U}_{k+1:k}$. Starting from a possibly correlated state $\rho_{SE}(t_0)$, the final reduced state is

$$\begin{aligned}\rho_S(t_n) &= \mathrm{Tr}_E\Big[\mathcal{U}_{n:n-1}(\mathcal{A}_{n-1}\otimes\mathcal{I}_E)\cdots\mathcal{U}_{1:0}(\mathcal{A}_0\otimes\mathcal{I}_E)[\rho_{SE}(t_0)]\Big] \\ &=: \mathcal{T}_{n:0}\big[\mathcal{A}_{n-1},\ldots,\mathcal{A}_0\big].\end{aligned} \tag{38}$$

The process tensor $\mathcal{T}_{n:0}$ is the higher-order map that remains after the initial joint state, the inaccessible environment, and all intermediate system–environment propagators have been absorbed into a single object [6–8]. A channel accepts a state and returns a state; a one-slot superchannel accepts an operation and returns a state or another operation. A process tensor (also referred to as quantum combs, commonly in the literature) is their multi-time extension: it accepts an ordered sequence of operations and returns the final state. If the final output is also measured or traced, the same object instead returns the associated probability or expectation value.

Despite acting on operations rather than directly on states, a physical process tensor inherits three familiar structural properties from quantum mechanics [6–8]. First, it is *linear in every intervention*. If an operation at time t_k is chosen as a probabilistic mixture $p\mathcal{A}_k + (1-p)\mathcal{B}_k$, then the resulting output is the same mixture of the outputs obtained from $\mathcal{A}_k$ and $\mathcal{B}_k$. For example,

$$\begin{aligned}&\mathcal{T}_{n:0}\big[\ldots, p\mathcal{A}_k + (1-p)\mathcal{B}_k, \ldots\big] \\ &\quad = p\,\mathcal{T}_{n:0}\big[\ldots,\mathcal{A}_k,\ldots\big] + (1-p)\,\mathcal{T}_{n:0}\big[\ldots,\mathcal{B}_k,\ldots\big].\end{aligned} \tag{39}$$

Second, the process tensor is *completely positive*. Inserting completely positive operations at its open times must produce a positive final state, even when those operations act jointly on the system and an arbitrary untouched ancilla. This is the multi-time counterpart of Eq. (31). It guarantees that the process remains physically meaningful when it is embedded inside a larger circuit, rather than only for the isolated sequence used to define it.

Finally, the process tensor is *causal*. A choice made at a later time cannot alter statistics already recorded at an earlier time. Operationally, once the future slots are filled with deterministic trace-preserving operations and their outputs are discarded, the reduced state (or outcome statistics) at time t_k cannot depend on which future operations were chosen:

$$\mathrm{Tr}_{>k}\,\mathcal{T}_{n:0}\big[\mathcal{A}_{n-1},\ldots,\mathcal{A}_0\big] = \mathcal{T}_{k:0}\big[\mathcal{A}_{k-1},\ldots,\mathcal{A}_0\big], \tag{40}$$

for every k and every pair of future CPTP fillings of the remaining slots. This no-signalling-from-the-future condition also supplies the appropriate multi-time notion of normalisation: every deterministic sequence must preserve total probability. In the Choi representation it

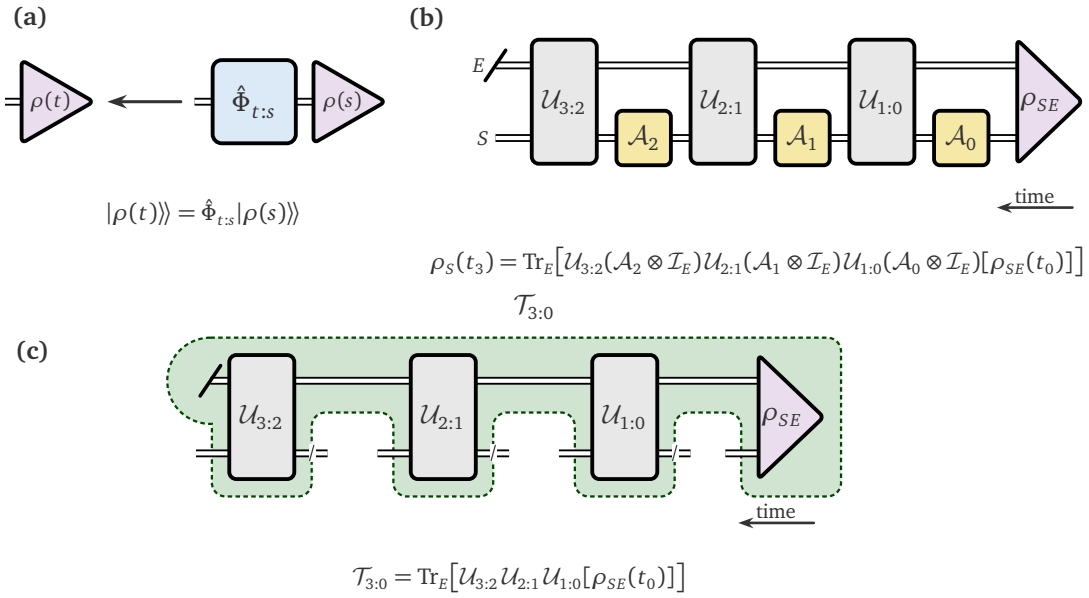

Figure 5: (a) A CPTP map $\hat{\Phi}_{t:s}$ takes a Liouville state $|\rho(s)\rangle\!\rangle$ to $|\rho(t)\rangle\!\rangle$. (b) Joint system–environment evolution with interventions $\mathcal{A}_k$ on the system only; both rails are Liouville wires, the environment is traced out, and the final system leg is left open. Time runs from right to left. (c) The same network with the interventions removed: the remaining object is the process tensor $\mathcal{T}_{3:0}$, with open Liouville slots at each intervention time.

becomes a hierarchy of partial-trace constraints, which we introduce in the following subsection. Together, linearity, complete positivity, and causality distinguish a physical time-ordered process from an arbitrary multilinear tensor with matching dimensions.

A point worth emphasising is that an individual trajectory selected from a process tensor need not be deterministic. An operation inserted at an intermediate time may represent a probabilistic event, which would then, reduce the trace of the outcome quantum state. Unit trace is determined by the choice of instruments, and so, trace-preserving trajectory is not a requirement for a process tensor. We return to this distinction in Sec. 4, where instruments are defined as families of completely positive maps that need not preserve the trace individually, while the complete experiment remains CPTP.

3.2 Choi representation, instruments, and process contraction

TODO: Fidel: add the content on the Choi representation of a PT, and connecting it with our theory.

3.3 Anatomy of a `ProcessTensor`

The Choi representation introduced above is complete, but its size grows exponentially with the number of intervention times. This is the temporal counterpart of the many-body problem encountered in Sec. 2: writing down every possible history as one dense tensor quickly becomes impractical. In this package, the process tensor is represented in matrix-product operator form. The environmental degrees of freedom are compressed into bond indices that carry only the influence of the environment that remains relevant to the system at each timestep. The outcome is an MPO whose physical indices are ordered in time rather than in space; we

write this stored object as $\Upsilon_{n:0}$, shown in Fig. 6,

$$\left[\Upsilon_{n:0}\right]_{\boldsymbol{o},\boldsymbol{i}} = \sum_{\chi_1,\ldots,\chi_{n-1}} [Q^{[n-1]}]^{\chi_{n-1}}_{o_{n-1}i_{n-1}} [Q^{[n-2]}]^{\chi_{n-1}\chi_{n-2}}_{o_{n-2}i_{n-2}} \cdots [Q^{[1]}]^{\chi_2\chi_1}_{o_1 i_1} [Q^{[0]}]^{\chi_1}_{o_0 i_0} \tag{41}$$

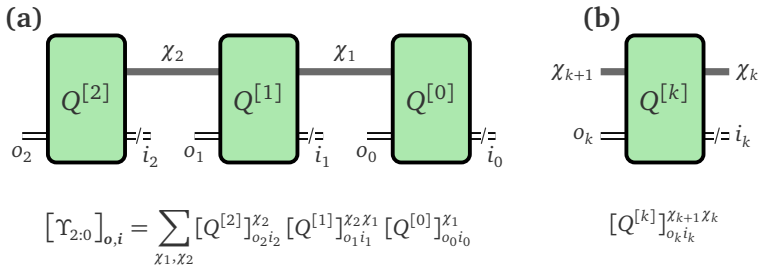

Figure 6: Matrix-product representation of a process tensor. (a) A three-step PT-MPO with time running from right to left: each core $Q^{[k]}$ carries a pair of double-wired system legs (output o_k, input i_k) and neighbouring cores are joined by a thick memory link χ_k. (b) Index anatomy of a generic interior core. Open system legs mark the times at which operations may be inserted, while the horizontal bonds transmit the compressed influence of the environment between different parts of the experiment.

Here i_k and o_k are the system input and output indices at time step k, while χ_k is an internal memory index joining neighbouring cores. The bond dimensions $D_k = \dim(\chi_k)$ quantify how large an effective space must be carried across each temporal cut. A Markovian process factorises across time and needs only trivial memory links, whereas a non-Markovian process generally has $D_k > 1$. A small value of D_k does not imply the absence of memory; it means that the relevant memory is compressible [6, 9].

To illustrate the indices that exist in our Process-Tensor MPO (PT-MPO), let us consider a three-step dense process tensor with one bath spin and a system spin (we will see how to construct this in the next subsection). The first core contains the following site indices:

Julia

```julia
siteinds(pt, 1)
```

Terminal

```
2-element Vector{Index{Int64}}:
 (dim=4|id=810|"Liouv,n=1,ptype=S=1/2,tstep=0")'
 (dim=4|id=810|"Liouv,n=1,ptype=S=1/2,tstep=0")
```

Listing 6: Site indices of the first PT-MPO core.

The primed index is i_0, the unprimed index is o_0, and their common `id` shows that they are two prime levels of the same ITensor index. The numerical ID is generated at runtime and can change from what is shown above.

The temporal memory indices are equally direct:

Julia

```julia
linkinds(pt)
```

Terminal

```
2-element Vector{Index{Int64}}:
 (dim=4|id=388|"Link,PT,tstep=1")
 (dim=4|id=209|"Link,PT,tstep=2")
```

Listing 7: Memory-link indices of the three-step process tensor.

These tags distinguish a PT-MPO from a conventional spatial MPO. A physical PT leg has `Liouv` (Liouville rather than Hilbert space), `ptype=S=1/2` (the underlying physical site type), and `tstep=k` (temporal position); its prime level distinguishes input from output. A memory bond has PT and `tstep=k` in addition to `Link`, replacing the usual spatial link position. Here each bond has dimension 4, the retained bath Liouville dimension; under compression the effective dimensions $D_k = \dim(\chi_k)$ may vary with k.

Unknown property access is delegated to `core`, so familiar tensor-network inspection functions can be applied directly to `pt`. For example, `linkdims(pt)` and `maxlinkdim(pt)` inspect the temporal memory bonds. The package-specific helpers `input_sites`, `output_sites`, and `coupling_times` then expose the exact temporal legs on which instruments will act. We postpone those contractions to the later sections.

3.4 Ingredients to construct a `ProcessTensor`

Having identified the anatomy of a `ProcessTensor`, we can now turn to the construction and ask what microscopic data are needed to construct this object. We introduce these inputs in the same order in which they define the physics: first the accessible system and its free Hamiltonian, then the environment together with its initial state, free Hamiltonian, and coupling to the system, and finally the discrete time grid on which the process is represented. Throughout, we use one system spin coupled to one bath spin for three time steps—the smallest running example that exhibits a non-trivial memory bond—so that each ingredient can be followed directly into the dense construction in the next subsection.

1. System: The first ingredient is the accessible system. The PT-MPO constructor infers the type of particle that the system represents from the site index used to construct it. In v0.3.0 of `ProcessTensors.jl`, we currently have support for `Spin-1/2`, `Spin-1`, `Qubit`, and `Boson` site-types for the system. The system's free Hamiltonian is also used by the constructor to append the free evolution of the system to the PT-MPO. In our small example, we use a single `Spin-1/2` system with the Hamiltonian

$$H_S = hS_S^x, \qquad h = 0.7. \tag{42}$$

It is constructed from an ordinary Hilbert-space spin index:

Julia

```julia
system_sites = siteinds("S=1/2", 1)

h = 0.7
H_S = OpSum()
H_S += h, "Sx", 1

system = spin_system(system_sites, H_S)
println(system)
```

Terminal

```
ProcessTensors.SpinSystem
  sites: 1
  space: Liouville
  site dims: 4
  dissipative: false
```

Listing 8: System Hamiltonian and `SpinSystem` constructor.

The first line identifies the model type. Although we supplied a Hilbert index of dimension 2, `spin_system` converted it internally into a Liouville index of dimension $2^2 = 4$; this accounts for the next three lines. The final line reports that no Lindblad jump operators were supplied. The present process-tensor builder requires a single-site system because the free-system propagation is embedded into every temporal core. An initial system state is intentionally absent: it is an experimental preparation and will later be inserted into the first open process-tensor leg as an instrument.

2. Environment: The next ingredient is the non-Markovian environment and its microscopic content that dictates how it influences the system. We store these microscopic content of the bath in an `AbstractBathMode`. An environment is built from a collection of individual bath modes in this package. In v0.3.0 of `ProcessTensors.jl`, we currently have support for `Spin-1/2`, `Spin-1`, and `Boson` site-types for the bath modes. The environment can also be empty. For each bath mode we need its Liouville site, local Hamiltonian, initial state, and coupling to the system. Our current implementation of PTs exclude the scenario where we start with initially correlated bath states. In our example, we take

$$H_E = \frac{\omega_E}{2} S_E^z, \qquad H_{SE} = g S_E^z S_S^z, \qquad \rho_E(0) = |\Downarrow\rangle\langle\Downarrow|, \tag{43}$$

with $\omega_E = 1.1$ and $g = 0.35$:

Julia

```julia
bath_sites = siteinds("S=1/2", 1)
bath_sites_L = liouv_sites(bath_sites)

omega_E = 1.1
H_E = OpSum()
H_E += (omega_E / 2), "Sz", 1

psi_E0 = MPS(bath_sites, ["Dn"])
rho_E0 = to_dm(psi_E0)
rho_E0_L = to_liouville(rho_E0; sites=bath_sites_L)

g = 0.35
H_SE = OpSum()
H_SE += g, "Sz", 1, "Sz", 2

mode = spin_mode(
    bath_sites_L,
    H_E,
    rho_E0_L;
    coupling=H_SE,
)
```

```julia
environment = spin_bath([mode])

println(environment)
```

Terminal

```
ProcessTensors.SpinBath
  modes: 1
  space: Liouville
  site dims: 4
  bath Liouville dimension: 4

  mode summary:
    [1] SpinMode(dim=4, H_terms=1, coupling_terms=1)
```

Listing 9: Bath Hamiltonian, initial state, coupling, and `SpinBath` constructor.

The two site labels in H_{SE} follow a local convention used by every independent bath mode: site 1 is the bath mode and site 2 is the system coupling site. They do not refer to two sites inside the system. This output first identifies a spin bath containing one mode. Its single Liouville site again has dimension 4, so the complete bath Liouville space also has dimension $D_E = 4$. The mode summary confirms that both the local bath Hamiltonian and the mode–system coupling contain one `OpSum` term. Notice that $\rho_E(0)$ was vectorised on the exact indices stored in `bath_sites_L`. Two Liouville indices with the same dimension and tags are not interchangeable if they have different ITensor identities.

3. Time grid: The last physical ingredient is the time grid,

$$t_k = k\Delta t, \qquad k = 0, \ldots, n_{\text{steps}}, \tag{44}$$

which is specified by

Julia

```julia
dt = 0.05
nsteps = 3
t_final = round(dt * nsteps; digits=10)
println("time grid: dt=$dt, nsteps=$nsteps, t_final=$t_final")
```

Terminal

```
time grid: dt=0.05, nsteps=3, t_final=0.15
```

Listing 10: Time grid for the dense process-tensor example.

Process labels count the stored intervention slots, not the number of time stamps. An `nsteps`$= N$ process contains the cores $Q^{[0]}, \ldots, Q^{[N-1]}$, tagged `tstep` $= 0, \ldots, N-1$, and is denoted $\Upsilon_{N-1:0}$; `t_final`$= N\,\Delta t$ is the endpoint of the last propagation interval. The present example is therefore `nsteps` $= 3 \iff Q^{[0]}, Q^{[1]}, Q^{[2]} \iff \Upsilon_{2:0}$, with $t_3 =$ `t_final` $= 0.15$. Fig. 7 summarises how these ingredients combine: a single-site system, an environment assembled from bath modes, and the chosen time grid.

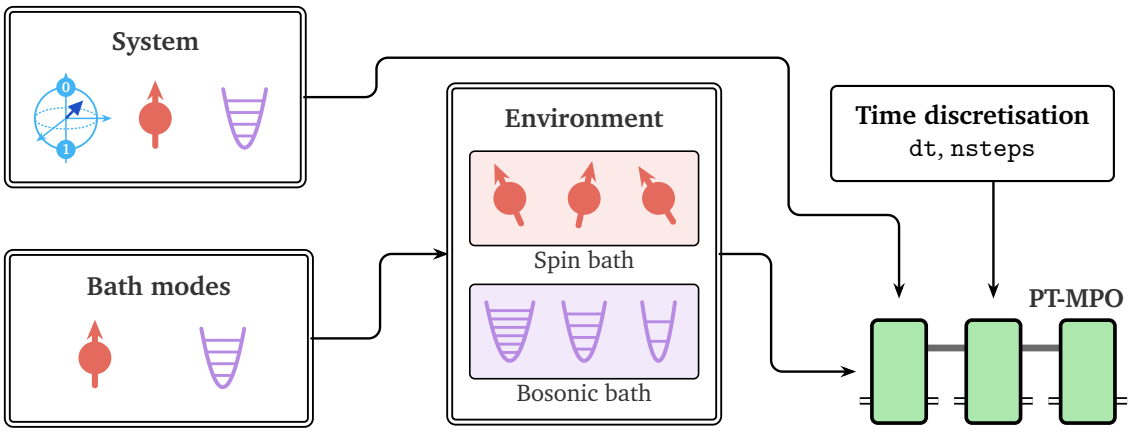

Figure 7: Microscopic ingredients of a `ProcessTensor`.

3.5 Builders to construct a `ProcessTensor`

The ingredients of Sec. 3.4 can be assembled by two builders that return the same `ProcessTensor` representation. `Dense()` retains the complete environmental Liouville space and suits one mode, a small multimode bath, or an exact reference calculation; `ACE()` adds independent modes sequentially and compresses the temporal memory after each addition.

For the one-spin environment introduced above, the complete joint Liouville space is still sufficiently small for the `Dense()` builder. To see what this method constructs, we return to the operational definition in Eq. (38). Between two consecutive intervention times, the system and environment evolve together under the one-step map

$$\mathcal{G}_{S+E}(\Delta t) = \exp\left[\Delta t\,(\mathcal{L}_S + \mathcal{L}_E + \mathcal{L}_{SE})\right]. \tag{45}$$

Constructing the process tensor amounts to repeating this map across the chosen time grid while leaving the system input and output legs at every step open. The initial environmental leg is contracted with the supplied state $|\rho_E(0)\rangle\!\rangle$, the environmental output of one step is connected to the environmental input of the next, and the final environmental leg is contracted with the trace effect $\langle\!\langle I_E|$. These contractions implement precisely the initialization and final partial trace appearing in Eq. (38); the system legs remain open because the corresponding preparations and interventions will be supplied later.

The environmental legs connecting successive propagators are the microscopic origin of the memory links introduced in Eq. (41). In a dense construction, they retain the complete bath Liouville space. A bath with Hilbert-space dimension d_E therefore produces uncompressed memory links of dimension d_E^2. For N_E modes with local dimensions d_m, the complete joint Liouville dimension is

$$D_{S+E}^{(L)} = d_S^2 \prod_{m=1}^{N_E} d_m^2, \tag{46}$$

which makes direct exponentiation practical only for a single mode or a small multimode environment.

Although Eq. (45) gives the complete physical propagator, the package does not construct it as one inseparable exponential. Instead, it writes

$$\mathcal{L}_{S+E} = \mathcal{L}_S + \mathcal{L}_{E+SE}, \qquad \mathcal{L}_{E+SE} := \mathcal{L}_E + \mathcal{L}_{SE}. \tag{47}$$

This separation reflects the architecture shared by the process-tensor builders. The bath and interaction generator $\mathcal{L}_{E+SE}$ constructs the environmental influence and its temporal memory

links, whereas $\mathcal{L}_S$ generates the deterministic free evolution of the accessible system. Free-system evolution does not itself create environmental memory, but it changes how the system interacts with that memory and must therefore remain part of the final process tensor. Accordingly, `ProcessTensors.jl` constructs the environmental slab first and embeds the free-system map into every PT-MPO core afterwards; the system evolution is separated during construction, not discarded.

Because $\mathcal{L}_S$ and $\mathcal{L}_{E+SE}$ need not commute, their propagators cannot generally be multiplied without approximation. Defining

$$\mathcal{M}_S(\tau) = \exp(\tau\mathcal{L}_S), \qquad \mathcal{Q}_{E+SE}(\tau) = \exp(\tau\mathcal{L}_{E+SE}), \tag{48}$$

the exact one-step map and the two available splittings are

$$\begin{aligned}\mathcal{G}_{S+E}(\Delta t) &= \exp\left[\Delta t\,(\mathcal{L}_S + \mathcal{L}_{E+SE})\right] \\ &\approx \begin{cases} \mathcal{Q}_{E+SE}(\Delta t)\,\mathcal{M}_S(\Delta t) & \text{for } \texttt{Trotter\{1\}()}, \\ \mathcal{M}_S(\Delta t/2)\,\mathcal{Q}_{E+SE}(\Delta t)\,\mathcal{M}_S(\Delta t/2) & \text{for } \texttt{Trotter\{2\}()}. \end{cases}\end{aligned} \tag{49}$$

For Hamiltonian evolution, $\mathcal{M}_S(\tau)$ is the unitary channel generated by $U_S(\tau) = \exp(-\mathrm{i}H_S\tau)$; when the system contains Lindblad terms, it denotes the corresponding free-system CPTP map. The symmetric choice reduces the local splitting error from $\mathcal{O}(\Delta t^2)$ to $\mathcal{O}(\Delta t^3)$, and hence the accumulated error at fixed final time from $\mathcal{O}(\Delta t)$ to $\mathcal{O}(\Delta t^2)$. It does not eliminate time-discretisation error, so physical results will be more accurate as Δt is decreased.

With this construction, the process tensor for the one-spin example is obtained by

Julia

```julia
using ITensors.Ops: Trotter

pt_dense = build_process_tensor(
    system;
    method=Dense(),
    environment=environment,
    dt=dt,
    nsteps=nsteps,
    sys_alg=Trotter{2}(),
    progress=false,
)

println(pt_dense)
```

Terminal

```
3-step ProcessTensor{SpinSystem, SpinBath} | dt=0.05 | t_final=0.15 |
 ↪  maxlinkdim=4

  system:      SpinSystem(nsites=1, dissipative=false)
  environment: SpinBath(nmodes=1, D_bath=4, coupling=true)
  core:        MPO{Liouville}(length=3, linkdims=[4, 4])
```

Listing 11: Dense construction of the one-spin-bath process tensor.

The three MPO tensors correspond to the three propagation intervals specified by `nsteps`. Each memory link has dimension $4 = 2^2$, confirming that the dense builder has retained the complete Liouville space of the single bath spin. No temporal compression has been introduced at this stage.

The dense product space becomes impractical for many microscopic modes. However, when the environment constructed in Sec. 3.4 is a collection of initially uncorrelated modes, each with its own Hamiltonian and system coupling but no direct mode–mode coupling, its generator separates as

$$\mathcal{L}_{E+SE} = \sum_{m=1}^{N_E} \mathcal{L}_{E_m+SE_m}, \tag{50}$$

This star-topology environment structure lets `ACE()` propagate each mode in its small Hilbert space, convert it to a closed influence PT-MPO, and absorb it sequentially without ever forming the full dense environmental operator.

After joining mode m, ACE compresses the temporal links by SVD:

$$\widetilde{\Upsilon}^{(m)}_{n:0} = \mathcal{C}_{\varepsilon,\chi_{\max}}\left[\widetilde{\Upsilon}^{(m-1)}_{n:0} \star \Upsilon^{(m)}_{n:0}\right], \tag{51}$$

Here $\star$ joins the accumulated and single-mode influences and ε (the `cutoff`) and $\chi_{\max}$ (the `maxdim`) are the singular value threshold and maximum bond dimension, respectively. When their generators do not commute, `combine_alg=Trotter{2}()` performs this join with symmetric second-order ordering. The map $\mathcal{C}_{\varepsilon,\chi_{\max}}$ then compresses each temporal cut with

$$\sigma_j > \varepsilon\sigma_1, \qquad \chi_k \leq \chi_{\max}. \tag{52}$$

Thus χ_k labels an effective temporal state rather than the complete microscopic bath, and ACE is efficient when its dimension remains small. A more detailed derivation and applications of the ACE alforithm can be found in Refs. [11, 14, 18].

Two compression schedules are available. The default `:canonzip` first joins a complete mode without truncation, canonicalises towards the final time, and performs a right-to-left relative-SVD sweep, allowing the retained basis to see the mode over the full interval. `:zipup` truncates during the forward join before a backward sweep, reducing intermediate sizes for long chains or tight memory budgets at the cost of greater cutoff sensitivity. Tree-like joining is another ACE strategy [27], but is not implemented here.

For a vector `modes` whose elements were constructed as in the preceding subsection, the many-mode process is built with

Julia

```julia
ace_environment = spin_bath(modes)

pt_ace = build_process_tensor(
    system;
    method=ACE(
        cutoff=1e-10,
        maxdim=512,
        compression=:canonzip,
    ),
    environment=ace_environment,
    dt=dt,
    nsteps=nsteps,
    sys_alg=Trotter{2}(),
    combine_alg=Trotter{2}(),
```

```
)
```

Listing 12: ACE construction for a bath of independent microscopic modes.

The cutoff is a convergence parameter, not a target observable accuracy: increasing it discards more directions, while a smaller `maxdim` may force truncation above the cutoff. Because generic SVD truncation need not preserve complete positivity or causal normalisation, aggressive compression can cause trace drift, small negative eigenvalues, or unstable observables. Convergence therefore requires smaller cutoffs and larger caps while monitoring trace, Hermiticity, minimum eigenvalue, and target observables.

4 Quantum instruments and multi-time experiments

Constructing a process tensor is only the first half of the story; the physics appears from it when we construct and contract the intermediate interventions of the system at each timestep. The process tensor constructed in the previous section is deliberately left as an abstract object with its system input and output legs open for later intervention. This is what makes the object reusable. The same PT-MPO can describe uninterrupted reduced dynamics, a controlled experiment, a post-selected measurement trajectory, or a multi-time correlation function; what changes between these calculations is not the process tensor, but the instruments contracted with its temporal legs. We first introduce these instruments in their strict operational sense, then explain the instrument interface and classification in `ProcessTensors.jl`, the lazy schedules that organise them, and the memory-bearing testers that correlate controls across time. The section concludes with two key functions that turn these ingredients into physical results: `evaluate_process` and `evolve`.

4.1 Operational meaning of instruments

A quantum operation describes one possible transformation of a system, including the possibility that the transformation occurs only for a selected measurement outcome. For an outcome x, it is a completely positive, trace-non-increasing map

$$\mathcal{A}_x : \mathcal{B}(\mathcal{H}_{\text{in}}) \longrightarrow \mathcal{B}(\mathcal{H}_{\text{out}}), \qquad \mathcal{A}_x(\rho) = \sum_\mu K_{x\mu} \rho K_{x\mu}^\dagger, \tag{53}$$

with

$$\sum_\mu K_{x\mu}^\dagger K_{x\mu} \leq \mathbb{I}. \tag{54}$$

Here, μ labels the unresolved Kraus components associated with that outcome. Complete positivity guarantees that

$$(\mathcal{A}_x \otimes \mathcal{I}_R)(X) \geq 0 \qquad \text{for every } X \geq 0 \tag{55}$$

and for an arbitrarily large untouched reference R. This requirement is essential in a multi-time setting: the system on which an intervention acts may already be correlated with an environment, an ancillary memory, or other degrees of freedom that are not part of the local description. The inequality in Eq. (54) then ensures that the trace of the unnormalised output can be interpreted as the probability of the selected event,

$$p(x) = \text{Tr}[\mathcal{A}_x(\rho)], \qquad \rho_x = \frac{\mathcal{A}_x(\rho)}{p(x)} \quad \text{when } p(x) > 0. \tag{56}$$

A quantum instrument is the complete family of such outcome maps,

$$\mathfrak{I} = \{\mathcal{A}_x\}_{x\in\mathsf{X}}, \qquad \sum_{x\in\mathsf{X}} \mathcal{A}_x \text{ is completely positive and trace preserving,} \tag{57}$$

or, equivalently,

$$\sum_{x,\mu} K_{x\mu}^\dagger K_{x\mu} = \mathbb{I}. \tag{58}$$

The selected map $\mathcal{A}_x$ is generally probabilistic and trace decreasing, while the non-selective operation $\sum_x \mathcal{A}_x$, obtained by forgetting the outcome, is deterministic and trace preserving. This distinction between an individual event and the complete experiment is the central operational content of the instrument formalism [28, 29].

Most familiar laboratory operations fit into this hierarchy. An initial preparation has no incoming quantum state and is formally represented as a map from the one-dimensional space $\mathbb{C}$ to the prepared state, $\mathcal{P}_{\rho_0}(\lambda) = \lambda\rho_0$, where λ is a scalar weight, normally equal to one for a deterministic preparation. A unitary pulse and a noise channel are deterministic two-sided maps from one system state to another. A measurement outcome x, by contrast, is represented by a positive operator E_x, called a quantum effect. The effects associated with all possible outcomes form a positive operator-valued measure (POVM),

$$E_x \geq 0, \qquad \sum_x E_x = \mathbb{I}, \tag{59}$$

so each E_x is referred to, henceforth, as a POVM element and satisfies $0 \leq E_x \leq \mathbb{I}$. For an input state ρ, this POVM element assigns the outcome probability

$$p(x) = \mathrm{Tr}(E_x\rho).$$

Combining the POVM element with a newly prepared state σ_x gives the measure-and-prepare operation

$$\mathcal{A}_x^{\mathrm{MP}}(\rho) = \mathrm{Tr}(E_x\rho), \sigma_x, \tag{60}$$

which explicitly breaks the system channel carrying information from the past while allowing the environment to retain a conditional memory of the outcome.

An observable should be distinguished from an instrument outcome. A positive effect E_x defines a physical event, whereas a general Hermitian observable $O = \sum_x o_x E_x$ defines the weighted combination

$$\langle O\rangle = \sum_x o_x p(x). \tag{61}$$

The direct insertion of O into a tensor-network contraction is therefore a convenient linear probe, not necessarily a completely positive outcome map. The package supports both uses: physical operations are used to describe experimental trajectories, while general operator insertions efficiently evaluate expectation values and correlation functions.

For a sequence of outcomes $\boldsymbol{x} = (x_0, \ldots, x_{N-1})$, the process tensor in Eq. (38) returns the unnormalised conditional state

$$\tilde{\rho}_S^{(\boldsymbol{x})}(t_N) = \mathcal{T}_{N:0}\big[\mathcal{A}_{x_{N-1}}^{(N-1)}, \ldots, \mathcal{A}_{x_0}^{(0)}\big], \qquad p(\boldsymbol{x}) = \mathrm{Tr}\,\tilde{\rho}_S^{(\boldsymbol{x})}(t_N). \tag{62}$$

Summing over every outcome at every time replaces the selected maps by deterministic CPTP operations and restores unit total probability. Thus, a process tensor itself is causally normalised, but a trajectory obtained from it need not have unit trace. This is precisely the distinction anticipated at the end of Sec. 3.1 and formalised by the operational process-tensor framework [6].

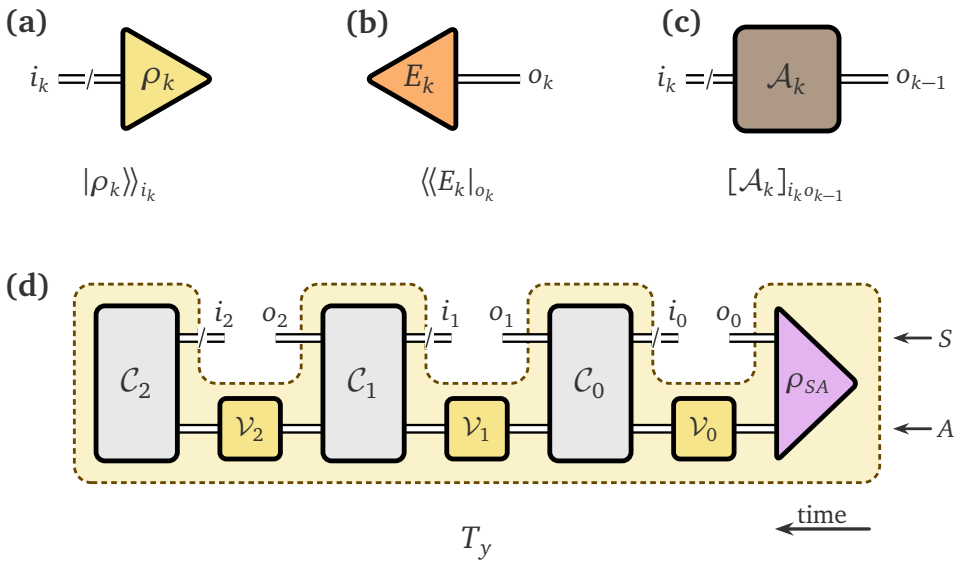

Figure 8: Anatomy of the instrument objects contracted with a PT-MPO. (a) An input-only preparation occupies the primed system index i_k. (b) An output-only effect occupies the unprimed index o_k. (c) A two-leg map joins o_{k-1} to i_k between adjacent slots. (d) A tester T_y: the joint initial state ρ_{SA} (purple triangle) occupies both rails at the earliest time; subsequent joint maps $\mathcal{C}_0, \mathcal{C}_1, \mathcal{C}_2$ span both rails, while ancilla-only operations $\mathcal{V}_k$ sit on the persistent A wire. The yellow dashed comb is the dual of Fig. 5(c), with open Liouville slots above the boundary. Note that the primed and unprimed legs appear in the opposite direction in a tester instrument to match the indices of the PT-MPO.

4.2 Instrument organisation in `ProcessTensors.jl`

The package uses the word *instrument* slightly more broadly than the strict definition in Eq. (57). An `AbstractInstrument` is any lazily specified tensor that can be inserted on the system legs of a `ProcessTensor`. This interface includes physical preparations and channels, but also observable insertions, arbitrary left–right actions, open-leg instructions, and user-supplied tensors. The common interface is computational: it tells the contraction how to materialise an object on the correct PT indices. As of v0.3.0, complete positivity and trace preservation are not inferred for an arbitrary custom tensor, so users who construct a physical protocol remain responsible for supplying physical maps.

Two abstract subtypes encode the first structural distinction: `SingleLegInstrument` and `TwoLegInstrument`. The first occupies one process-tensor leg and therefore begins or terminates a system wire. The second joins an output of one PT core to the input of the next and acts as a map between adjacent intervention slots. Using the notation of Eq. (41), the unprimed system index o_k is the output of the core $Q^{[k]}$, whereas the primed index i_k is its input. Their ITensor tags contain the same `tstep=k`; their prime levels distinguish the two roles,

$$\mathrm{plev}(o_k) = 0, \qquad \mathrm{plev}(i_k) = 1. \tag{63}$$

A schedule entry at logical step $k \geq 1$ connects o_{k-1} to i_k,

$$[\mathcal{A}_k]_{i_k o_{k-1}}, \tag{64}$$

while the initial preparation and terminal effect have only one leg,

$$[\mathcal{P}_{\rho_0}]_{i_0} = |\rho_0\rangle\!\rangle_{i_0}, \qquad [\mathcal{M}]_{o_{N-1}} = \langle\!\langle M|_{o_{N-1}}. \tag{65}$$

Constructor	PT legs	Role in a contraction
`state_preparation(`ρ`)`	input	Initial or replacement state
`observable_measurement(`O`)`	output/input	Effect or general observable insertion
`trace_out()`	output	Deterministic identity effect
`identity_operation()`	output–input	Identity channel between adjacent slots
`unitary_propagation(`H`, sites)`	output–input	Control generated by a Hamiltonian; $H(t)$ is also accepted
`left_right_operator(`A, B`)`	output–input	Linear map $\rho \mapsto A\rho B$
`left_action`, `right_action`	output–input	Convenient one-sided operator actions
`ProductInstrument`	separate pair	Output effect multiplied by an input preparation
`custom_twoleg_instrument(...)`	output–input	User-supplied dense Liouville map
`open_output()`, `open_input()`	one open leg	Keep a selected output or input unresolved
`open_inout()`	two open legs	Keep both legs unresolved and independent

Table 2: Principal lazy instrument constructors in `ProcessTensors.jl`. "PT legs" specifies which temporal indices the materialised object occupies.

Figure 8 collects these cases and places them beside the persistent ancillary wire used by a tester.

The single-leg types are intentionally restrictive where the physics fixes their orientation. `StatePreparation` acts only on an input leg, and `TraceOut` acts only on an output leg by inserting the vectorised identity effect $\langle\langle \mathbb{I}|$. Likewise, `OpenInput` and `OpenOutput` are fixed to prime levels one and zero, respectively. `ObservableMeasurement` targets an output leg by default, but may be placed on an input leg with `leg_plev=1`; the latter is useful when an operator must act from the right in a correlation-function schedule. The two-leg constructors validate the neighbouring-time rule in Eq. (64) whenever concrete PT sites are supplied.

Table 2 gives the compact constructor dictionary needed for the remainder of the paper. These constructors store Hilbert-space states, Hamiltonians, observables, or maps without immediately producing a dense tensor. The corresponding Liouville-space `ITensor` is created only after a schedule is bound to a specific `ProcessTensor` in a contraction function. Several physicality distinctions in Table 2 are worth retaining. `identity_operation()` and unitary propagation are CPTP. A normalised density operator defines a deterministic preparation, and a positive effect bounded by the identity defines a probabilistic measurement outcome. By contrast, the general map $\rho \mapsto A\rho B$ is not necessarily completely positive, an arbitrary observable is not necessarily an effect, and a `CustomTwoLegInstrument` is not automatically checked for complete positivity or trace preservation. Even though these objects need not correspond to an operational instrument, they are still useful to compute physically relevant quantities such as multi-time correlation functions. A `ProductInstrument` is produced by multiplying an output-side and an input-side single-leg object; for a POVM measurement and a normalised preparation it realises the causal break in Eq. (60).

`open_in, open_out, open_inout` constructors are bookkeeping operations rather than physical maps to keep track of which legs we wish to leave uncontracted by the end of a process evaluation. They materialise as the scalar tensor `ITensor(1.0)` during contraction, deliberately leaving the claimed PT indices uncontracted. They are different from `identity_operation()`. The former retains two independent indices, whereas the latter contracts them through an identity channel. Selective opening is important because `evaluate_process` returns a dense tensor whenever several system legs remain. If m Liouville legs of a d-dimensional system are

open, the dense object contains

$$d^{2m} \tag{66}$$

complex entries. For a qubit, sixteen open legs already require 4^{16} complex numbers, or 64 GiB in `ComplexF64` before tensor overhead. Open-leg schedules should therefore expose only the temporal indices required by the calculation.

4.3 Assembling multi-time experiments using `InstrumentSeq`

An experiment rarely contains only one operation. `InstrumentSeq` is the lazy, time-indexed dictionary that records which instrument is inserted at every logical timestep. Its three fields are

$$\texttt{InstrumentSeq} = (\texttt{default}, \texttt{entries}, \texttt{nsteps}). \tag{67}$$

The `default` fills ordinary slots for which no explicit action is given; `entries` is a sparse dictionary of overrides; and `nsteps` sets the upper time-step limit within which the instruments will act. The initial slot `tstep=0` is exceptional: it has no default because an initial system state is not part of the process tensor, and only a state preparation may be added there. The terminal slot `tstep=pt.nsteps` acts on the final output o_{N-1}. Consequently, an N-core process materialises $N+1$ instrument tensors: one preparation, $N-1$ intermediate operations, and one terminal action.

The three-step process `pt_dense` from Sec. 3.5 can be probed by preparing an up spin, applying an additional control Hamiltonian during the second bridge, and reading out S^z at the final output:

Julia

```julia
psi0 = MPS(system_sites, ["Up"])
rho0 = to_dm(psi0)

H_control = OpSum()
H_control += 0.4, "Sy", 1

O_final = OpSum()
O_final += 1.0, "Sz", 1

seq = InstrumentSeq(
    default=identity_operation(),
    nsteps=pt_dense.nsteps,
)
seq += state_preparation(rho0), 0
seq += unitary_propagation(H_control, system.sites), 2
seq += observable_measurement(O_final), pt_dense.nsteps
```

Listing 13: A lazy instrument schedule for a controlled three-step experiment.

This incremental += syntax resembles the use of `OpSum` in `ITensorMPS.jl`, but each tuple assigns an instrument to a time slot rather than adding an operator term.

The schedule in Listing 13 corresponds to the scalar contraction

$$z = \langle\langle S^z|_{o_2} Q^{[2]} \mathcal{U}_c Q^{[1]} \mathcal{I} Q^{[0]} |\rho_0\rangle\rangle_{i_0}, \tag{68}$$

where the tensor contraction over the temporal memory indices χ_1, χ_2 and the joined system indices is implicit. The default identity supplies $\mathcal{I}$ at step 1, the explicit entry at step 2 supplies $\mathcal{U}_c$, and the terminal observable closes the last output. Replacing the terminal observable by

`open_output()` turns the same history into a final reduced state; changing an intermediate entry to a measurement and preparation pair implements a causal break. The PT-MPO itself is unchanged in all three experiments.

During materialisation, consecutive single-leg output and input entries form their two-leg product. This allows a measurement at one side of a temporal cut and a preparation at the other to be written separately while retaining the product structure in Eq. (60). Conversely, incompatible placements are rejected before the costly contraction begins: an unpaired physical input action, a two-leg object at the terminal slot, or a timestep beyond `nsteps` produces an informative `ArgumentError` rather than a malformed tensor network.

4.4 Memory-bearing instruments: process testers

An `InstrumentSeq` describes a product schedule whose entries are chosen independently at different intervention times. A *process tester* is the more general multi-time experimental apparatus inserted into the open system legs of a process tensor [30, 31]. It may contain an ancillary quantum system A that is initialised once and retained throughout the experiment, allowing information acquired from the system at one intervention to influence controls applied at a later time.

A constructive realisation of such a tester is specified by

$$\mathbf{T} = \big(\rho_{SA},\ \{\mathcal{C}_k^{SA}\}_{k=0}^{N-1},\ \{\mathcal{V}_k^{A}\}_{k=0}^{N-1}\big), \tag{69}$$

where ρ_{SA} is the initial joint system–ancilla state,

$$\mathcal{C}_k^{SA} : \mathcal{B}(\mathcal{H}_S \otimes \mathcal{H}_A) \longrightarrow \mathcal{B}(\mathcal{H}_S \otimes \mathcal{H}_A) \tag{70}$$

is a joint system–ancilla operation, and

$$\mathcal{V}_k^{A} : \mathcal{B}(\mathcal{H}_A) \longrightarrow \mathcal{B}(\mathcal{H}_A) \tag{71}$$

acts only on the retained ancilla. These ingredients correspond directly to Fig. 8(d): ρ_{SA} enters at the right boundary, the maps $\mathcal{C}_k^{SA}$ span the system and ancillary rails, and the maps $\mathcal{V}_k^{A}$ act on the persistent ancillary wire as $\mathcal{I}_S \otimes \mathcal{V}_k^{A}$. Numerically, the former become two-site Liouville tensors and the latter one-site tensors, while the same ancillary index is contracted forward through the complete network. The system legs between successive joint maps remain available for contraction with the PT-MPO and therefore carry the opposite prime-level orientation to the matching process-tensor legs.

Because the ancillary degree of freedom is not reset between interventions, the Choi tensor of a correlated tester generally does not factorise into time-local operations,

$$\mathbf{T} \neq \bigotimes_{k=0}^{N-1} \widehat{\mathcal{A}}_k. \tag{72}$$

The product form is recovered when the ancilla is trivial or is independently reset after every intervention. An outcome-valued tester is obtained by terminating the network with a POVM $\{F_y\}$,

$$F_y \geq 0, \qquad \sum_y F_y = \mathbb{I}_{SA}, \qquad p(y|\Upsilon) = \mathrm{Tr}\big[F_y\, \omega_{SA}(\Upsilon)\big], \tag{73}$$

where $\omega_{SA}(\Upsilon)$ is the final state produced after contracting the memory-bearing network with the process Υ. Absorbing this sequential realisation and its terminal POVM into Choi operators gives the process-POVM elements [30, 32]

$$T_y \geq 0, \qquad p(y|\Upsilon) = T_y \star \Upsilon, \qquad \sum_y T_y = T_{\mathrm{det}}, \tag{74}$$

where T_{det} is the causally normalised deterministic tester. Its sequential realisation uses CPTP maps $\mathcal{C}_k^{SA}$ and $\mathcal{V}_k^{A}$; an individual outcome branch may instead be CP and trace non-increasing, provided that the complete set of branches sums to a CPTP deterministic strategy.

`ProcessTensors.jl` represents Eq. (69) using two objects. A `Tester` stores the ancillary site and its initial state, whereas a `TesterSeq` stores the ancillary-only and joint actions applied at each timestep. System-only controls remain in the ordinary `InstrumentSeq`; in the current API, the initial state is assembled as $\rho_{SA} = \rho_S \otimes \rho_A$ from the two objects.

Julia

```julia
system_sites = siteinds("Qubit", 1)
tester_sites = siteinds("Qubit", 1)
rho_A0 = to_dm(MPS(tester_sites, ["1"]))
memory = tester(tester_sites, rho_A0)

U_SA = op("SWAP", only(system_sites), only(tester_sites))
U_A = op("X", only(tester_sites))
joint_U_SA = joint_unitary(U_SA, system_sites, tester_sites)
local_U_A = tester_unitary(U_A, tester_sites)

tester_seq = TesterSeq(nsteps=pt_dense.nsteps)
tester_seq += joint_U_SA, 1
tester_seq += local_U_A, 2
tester_seq += joint_U_SA, 3
```

Listing 14: A single-qubit tester with reusable joint and local unitaries.

The site ordering of `U_SA` is system then tester, matching `vcat(system_sites, tester_sites)`. The joint unitary at steps 1 and 3 couples the system and ancilla, while the local unitary at step 2 acts only on the retained ancilla memory. Unspecified timesteps use `tester_identity()`. The current implementation supports one tester site and joint actions on one system site and one tester site. Tester actions are kept in `TesterSeq` and cannot be added to an `InstrumentSeq`, preventing the two schedules from being mixed.

4.5 Evaluating and evolving a process tensor

The objects introduced above are completed by `evaluate_process`. Given a PT-MPO Υ and a schedule $\mathbf{A}$, the function validates the claimed temporal legs, materialises each lazy instrument on the corresponding PT sites, optionally compiles the tester memory wire, and contracts the resulting network chronologically. Abstractly,

$$R_{\mathbf{A}} = \text{Contr}\big[\Upsilon, \mathcal{P}_{\rho_0}, \mathcal{A}_1, \ldots, \mathcal{A}_{N-1}, \mathcal{M}_N\big], \tag{75}$$

where the nature of $R_{\mathbf{A}}$ is determined by the system indices that the schedule leaves unresolved. Figure 9 shows the three return cases and the related sequence of snapshot contractions used by `evolve`.

A result with several open legs can be interpreted mathematically as a reduced or partially contracted process tensor, but its present Julia type is a dense `ITensor`, not a `ProcessTensor`. This distinction matters because the returned object no longer carries the PT-MPO metadata or temporal compression of the original process. A fully contracted result is a probability only when the terminal and intermediate insertions form a physical instrument; with a general observable or left–right action, the same scalar may instead be an expectation value or correlation function.

Open system legs	Julia return type	Interpretation
0	`ComplexF64`	Probability, trace, expectation value, or correlation scalar
1	`MPS{Liouville}`	Reduced state-like Liouville vector
2 or more	`ITensor`	Dense partially contracted multi-time object

Table 3: Return type of one `evaluate_process` call. A batch of fully contracted schedules returns one `Vector{ComplexF64}`.

The function `open_leg_info` predicts which input and output times remain unresolved before a contraction is attempted. In the following schedule, `open_inout()` at step 2 leaves o_1 and i_2 independent while the initial preparation and final trace close the remaining boundary legs:

Julia

```julia
seq_open = InstrumentSeq(
    default=identity_operation(),
    nsteps=pt_dense.nsteps,
)
add!(seq_open, state_preparation(rho0), 0)
add!(seq_open, open_inout(), 2)
add!(seq_open, trace_out(), pt_dense.nsteps)

info = open_leg_info(pt_dense, seq_open)
reduced = evaluate_process(pt_dense, seq_open; progress=false)

println("open input tsteps: ", info.open_in)
println("open output tsteps: ", info.open_out)
println("expected open legs: ", info.n_open_expected)
println("open dimensions: ", info.open_dims)
println("return type: ", typeof(reduced))
```

Terminal

```
open input tsteps: [2]
open output tsteps: [1]
expected open legs: 2
open dimensions: [4, 4]
return type: ITensor
```

Listing 15: Inspecting the open legs before constructing a dense reduced process.

This diagnostic turns the scaling in Eq. (66) into an immediate memory estimate. For example, the dimensions in Listing 15 predict a 4×4 dense object before the PT contraction begins. The helper also reports missing input or output coverage, allowing malformed schedules to be detected independently of the eventual return type.

The essential software advantage is now visible. Once `pt_dense` or an ACE-compressed PT has been constructed, different sequences can be evaluated without rebuilding the environmental influence. A terminal trace tests normalisation, a terminal observable gives an expectation value, a selected positive effect gives an outcome probability, an intermediate measurement–preparation pair gives a conditional trajectory, and selected open legs expose a

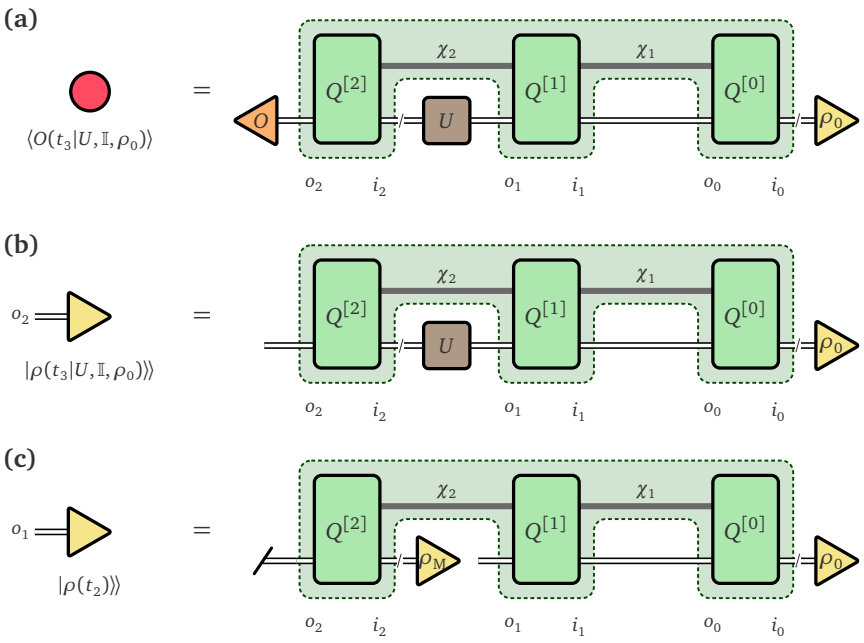

Figure 9: Evaluation and evolution of the instrument sequence in Listing 13. The dashed comb is the process tensor, while $|\rho_0\rangle\rangle$, $\mathbb{I}$, U, and $\langle\langle O|$ are the instruments that sit on the open system legs. (a) `evaluate_process` with an expectation of an observable: every system leg is closed and the process returns the conditional scalar $\langle O(t_3|U,\mathbb{I},\rho_0)\rangle$. (b) The same history with `open_output()` at the last slot: o_2 remains unresolved and the process returns the conditional Liouville state $|\rho(t_3|U,\mathbb{I},\rho_0)\rangle\rangle$. (c) An `evolve` snapshot at t_2: past controls are retained, the output o_1 is left open, and the unused future core $Q^{[2]}$ is closed by a trace-out on o_2 and a preparation $|\rho_M\rangle\rangle^{i_2} = |\mathbb{I}/d\rangle\rangle^{i_2}$.

reduced map. The same mechanism also supports the multi-time left–right insertions used by correlation functions.

The tester of Sec. 4.4 is contracted by the same function. Recalling Listing 14, `memory` carries the ancillary initial state and `tester_seq` stores the joint and local unitaries; `seq` remains the system schedule of Listing 13. Passing both objects into `evaluate_process` compiles the persistent A wire of Fig. 8(d) and traces it out after the final interval, so the return type is still fixed by the open *system* legs:

Julia

```
result = evaluate_process(
    pt_dense,
    seq;
    tester=memory,
    tester_seq=tester_seq,
    progress=false,
)
```

Listing 16: Evaluating the process tensor against the tester of Listing 14.

For the common special case in which the desired output is the reduced system state at every stored timestep, repeatedly assembling schedules by hand would be unnecessary. The `evolve` interface performs these contractions and returns both Liouville- and Hilbert-space

trajectories. A custom `InstrumentSeq` may be supplied in place of `rho0` alone when the trajectory includes control operations:

Julia

```julia
trajectory = evolve(pt_dense, seq; progress=false)

times = trajectory.times
rho_liouville = trajectory.states_liouville
rho_hilbert = trajectory.states_hilbert
```

Listing 17: Reduced dynamics from the process tensor and the instrument sequence of Listing 13.

The returned arrays contain one snapshot for each stored PT core, ordered by the logical times in `trajectory.times`. Later instruments in `seq` cannot affect an earlier reduced state since they are protected by the causality (no-signalling-from-the-future) property of the process tensor, Eq. (40) in Sec. 3.1. With a tester present, `states_liouville` and `states_hilbert` still contain the reduced system states; `tester_states_{liouville,hilbert}` are returned by default, and the joint fields `joint_states_{liouville,hilbert}` can be requested with `return_joint=true`.

Figure 9 depicts the schedule for one snapshot. The past preparation and controls are retained, the required output is left open, and every later core is discarded through the disconnected CPTP closure

$$\langle\langle \mathbb{I}|_{o_j} \otimes |\mathbb{I}/d\rangle\rangle^{i_{j+1}}. \tag{76}$$

The reference option `contraction=:evaluate` constructs precisely this full schedule and calls `evaluate_process` once per snapshot, giving an $O(N^2)$ contraction in the number of PT cores. The default `contraction=:closures` evaluates the same trajectory with one backward sweep of future closures followed by one forward prefix contraction, reducing the timestep scaling to $O(N)$. For a compressed PT, these backward closure tensors are essential: a temporal bond χ_k is an effective memory basis and cannot in general be terminated by inserting a bare bath trace vector.

5 Examples of process-tensor workflows

With the theory and basic programming interface of process tensors laid out in the previous sections, we can now look at some clean physical examples that showcase the application of our package. We will demonstrate how does the reduced system evolve, what is the probability of a multi-round experimental record, how are operators at different times correlated, and what changes when the intervention itself carries a persistent quantum memory? The physical examples below reproduce or adapt established benchmarks and protocols from the open-quantum-systems and quantum-information literature. Our purpose is to show that they can all be expressed by changing the instruments contracted with a common `ProcessTensor` representation. Since the complete walkthrough of the below examples is already provided in the package documentation, we only provide a small physical motivation of the problem, and show the core part of the code that produces the figures for each subsection. For the full codes, refer to the process tensors examples in the package documentation.

5.1 Reduced dynamics of a central spin

A first use of a process tensor is the familiar "recover the reduced dynamics of a system after its environment has been eliminated". We borrow, for this purpose, the fully polarised central-spin benchmark used by Cygorek *et al.* to demonstrate ACE for a non-Gaussian spin environment [11]. A central spin-1/2, $\mathbf{S}$, interacts isotropically with N noninteracting bath spins $\mathbf{s}_k$,

$$H_S = H_E = 0, \qquad H_{SE} = \sum_{k=1}^{N} \frac{J_k}{\hbar^2}\, \mathbf{S}\cdot\mathbf{s}_k, \qquad J_k = \frac{J}{N}. \tag{77}$$

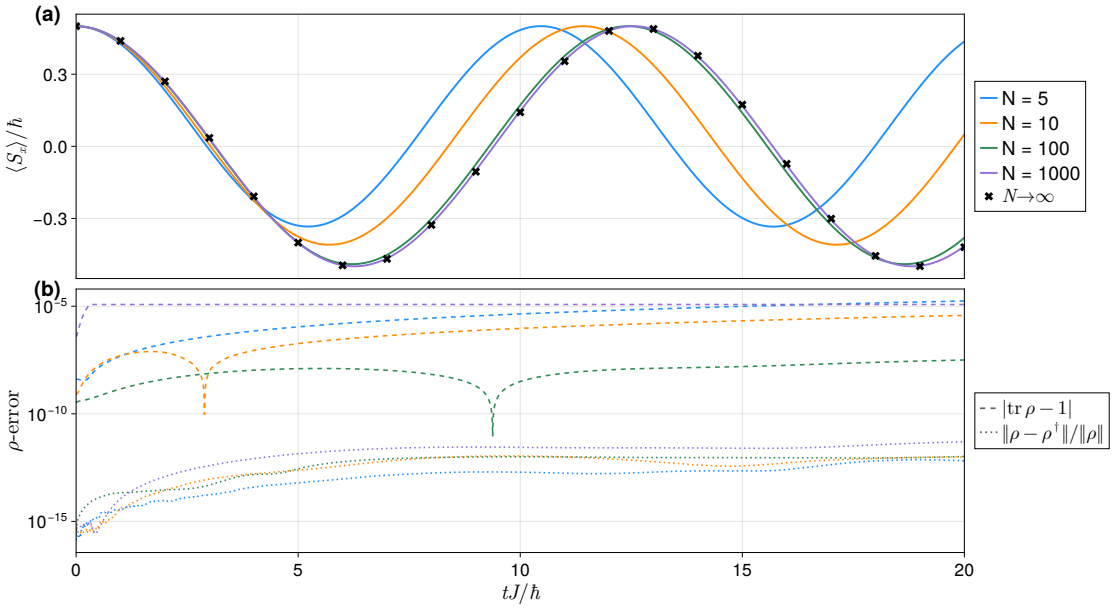

Figure 10: **(a) Fully polarised central-spin dynamics.** Reduced dynamics of the system spin coupled to a finite number of bath spins. Finite baths of the model in Eq. (77) approach the analytical $N \to \infty$ precession in Eq. (78) as N increases. **(b) Trace and hermiticity errors of ρ.** The lower panel records $|\mathrm{Tr}\,\rho - 1|$ and $\|\rho - \rho^\dagger\|/\|\rho\|$.

The system is initially polarised along $+x$, while every bath spin begins along $+z$. Keeping $\sum_k J_k = J$ fixed prevents the interaction energy scale from growing with the number of modes. The system and bath spins have no free Hamiltonian that leads to a unitary evolution in their own subspace. The effects we will se of the bath, will be purely because of the initial condition we choose for its spin modes. When we have just a few bath spins, we might observe non-trivial dynamics where a spin mode in the bath can couple with the central spin and produce non-trivial coupling with another spin mode in the same bath, despite no real coupling existing between the two bath spins. However, in the limit $N \to \infty$, the bath experiences vanishing back-action and produces a static effective magnetic field, giving

$$\frac{\langle S_x(t)\rangle}{\hbar} \longrightarrow \frac{1}{2}\cos\!\left(\frac{Jt}{2\hbar}\right). \tag{78}$$

Each bath spin is supplied to `ACE()` as an independent `SpinMode`. Once their joint influence has been compressed into a PT-MPO, the microscopic bath is no longer needed for the reduced calculation. Figure 10 shows the finite-bath oscillations and their approach to the collective large-N response.

Julia

```julia
bath_sites = siteinds("S=1/2", N)
bath_sites_L = liouv_sites(bath_sites)

modes = SpinMode[]
for k in 1:N
    rho_k = to_dm(MPS([bath_sites[k]], ["Up"]))
    coupling = OpSum()
    coupling += J / N, "Sx", 1, "Sx", 2
    coupling += J / N, "Sy", 1, "Sy", 2
    coupling += J / N, "Sz", 1, "Sz", 2
    push!(
        modes,
        spin_mode([bath_sites_L[k]], OpSum(), rho_k;
        ↪ coupling=coupling),
    )
end
bath = spin_bath(modes)

process_tensor = build_process_tensor(
    system; method=ACE(cutoff=1e-10),
    environment=bath, dt=0.01, nsteps=nsteps,
)

trajectory = evolve(process_tensor, rho_S0)
Sx_t = [real(tr(Sx * density_matrix(rho)))
        for rho in trajectory.states_hilbert]
```

Listing 18: Independent bath spins assembled into an ACE process tensor.

5.2 Repeated Ramsey measurements with active reset

Reduced dynamics observes the system without intervening. A process tensor also predicts the joint statistics of an experiment in which measurements and controls are applied repeatedly. We illustrate this using Ramsey interferometry, introduced originally as a separated-oscillatory-field spectroscopy protocol [33], and now ubiquitous in qubit characterisation. This choice is also motivated by recent studies of temporally correlated circuit noise generated by multi-mode spin–boson environments [34,35]. The qubit is coupled through Z to a thermal bosonic environment described by the pure-dephasing spin–boson model,

$$H_E = \sum_k \omega_k b_k^\dagger b_k, \qquad H_{SE} = Z \sum_k \sqrt{J(\omega_k)}(b_k + b_k^\dagger), \qquad J(\omega_k) = 2\alpha\omega_k e^{-\omega_k/\omega_c}, \tag{79}$$

which is a standard model of structured dephasing noise [36]. The continuum is discretised into thermal oscillator modes and compressed once with `ACE()`.

The experimental protocol we perform in the qubit is the following: the qubit is initially prepared in $\rho_r = |+\rangle_x\langle+|$, evolves for a fixed Ramsey time, and is read out with the POVM

$$E_x^{(\eta)} = \frac{1}{2}\big(\mathbb{I} + x\eta\sigma_x\big), \qquad x \in \{-1,+1\}, \qquad 0 \le \eta \le 1. \tag{80}$$

Here, η is the effective measurement visibility (sharpness), quantifying how reliably the Ramsey readout distinguishes the two X-basis outcomes. A POVM element fixes the outcome probability but not the state supplied to the following round. We therefore combine the readout

with an outcome-independent active reset and use the causal-break operation

$$\mathcal{A}_x^{\mathrm{MR}}(\rho) = \mathrm{Tr}\big[E_x^{(\eta)}\rho\big]\rho_{\mathrm{r}}, \qquad [\mathcal{A}_x^{\mathrm{MR}}] = |\rho_{\mathrm{r}}\rangle\!\rangle\langle\!\langle E_x^{(\eta)}|. \tag{81}$$

Each outcome map is CP and trace-non-increasing, while their sum is the CPTP replacement channel

$$\sum_x \mathcal{A}_x^{\mathrm{MR}}(\rho) = \mathrm{Tr}(\rho)\rho_{\mathrm{r}}. \tag{82}$$

Measure-and-reprepare causal breaks are central to the operational definition and experimental reconstruction of non-Markovian processes [6, 37, 38]. A causal break introduced in the system will ensure no information of the qubit's previous states before this break affects its future dynamics through the qubit itself. If any correlation is observed in the following dynamics, it should come from the process tensor of the environment.

`ProcessTensors.jl` does not yet provide a dedicated POVM container in v0.3.0. Each outcome map in Eq. (81) is instead assembled from two single-leg instruments: an output-leg `observable_measurement` of $E_x^{(\eta)}$ and an input-leg `state_preparation` of ρ_{r}. The package overloads `*` for this purpose. Provided the two instruments have opposite prime levels—one with plev $= 0$ and one with plev $= 1$—their product is a `ProductInstrument`, a two-leg object inserted at a single evolve slot of an `InstrumentSeq`. If we perform this POVM measurement and reset three times, we obtain the joint probability $p(x_1, x_2, x_3)$ of the record $|x_1\rangle_x, |x_2\rangle_x, |x_3\rangle_x$.

Julia

```julia
eta = 0.90
rho_r = to_dm(MPS(system_sites, ["+"]))

effects = Dict(x => (OpSum() + (0.5, "Id", 1) + (0.5 * x * eta, "X", 1)) for x in (-1, 1))
instruments = Dict(
    x => observable_measurement(effects[x]) * state_preparation(rho_r)
    for x in (-1, 1)
)

for record in Iterators.product((-1, 1), (-1, 1), (-1, 1))
    seq = default_schedule(process_tensor)
    seq += state_preparation(rho_r), 0
    for (k, x) in zip(readout_steps, record)
        seq += instruments[x], k
    end
    seq += trace_out(), process_tensor.nsteps
    probabilities[record] = real(evaluate_process(process_tensor, seq))
end
```

Listing 19: POVM-and-reset `ProductInstruments` for three Ramsey readouts.

Evaluating the process tensor for each of the above records, the returned scalar is the joint probability

$$p(x_1, x_2, x_3) = \mathrm{Tr}\Big[\mathcal{T}\big[\mathcal{A}_{x_3}^{\mathrm{MR}}, \mathcal{A}_{x_2}^{\mathrm{MR}}, \mathcal{A}_{x_1}^{\mathrm{MR}}, \mathcal{P}_{\rho_{\mathrm{r}}}\big]\Big]. \tag{83}$$

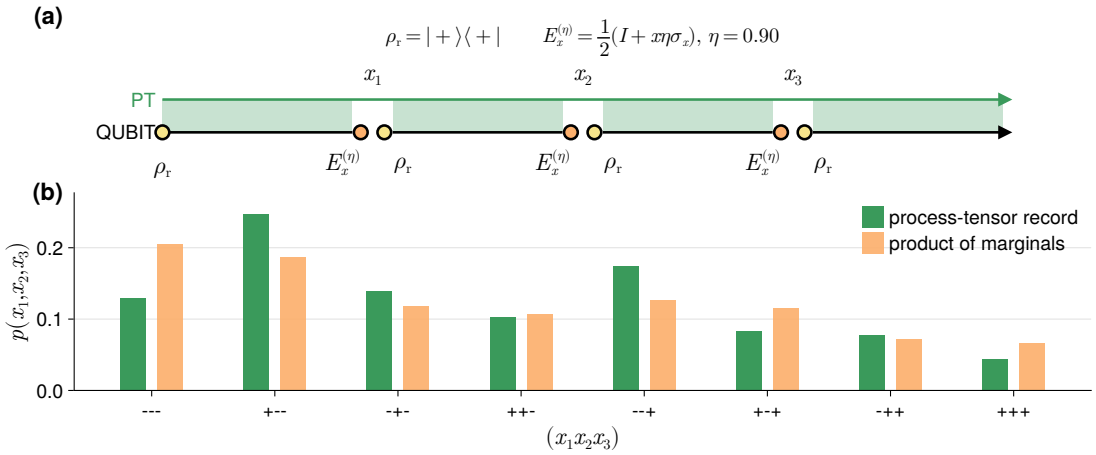

Figure 11: **(a) Timeline of the Ramsey readout protocol.** Three repeated readouts of the thermally dephasing qubit in Eq. (79), each followed by the same active reset, Eq. (81). The time exposed to the environment between each prepare and measure is the same for all three readouts. The green fill between the PT and QUBIT line depicts constant interaction between them. **(b) Outcome recorded probabilities** $p(x_1, x_2, x_3)$**.** For some three-shot records the process-tensor probabilities do not coincide with the products of the single-round marginals, indicating correlations between successive readouts.

The eight probabilities in Fig. 11 are non-negative and sum to unity, as required by Eq. (82). Since the time for which the qubit is exposed to the noisy environment between each prepare and measure is the same for all three readouts, we would expect the joint probability $p(x_1, x_2, x_3)$ to be the product of the single-round probabilities $p(x_1)p(x_2)p(x_3)$. Any resolved deviation from this product reveals that the environment retained information about an earlier measurement and returned it after the system itself had been reset. Thus, this is an operational signature of non-Markovianity.

5.3 Multi-time correlations in a spin environment

Correlation functions at multiple times govern response theory, noise spectroscopy, and multidimensional spectroscopy. In a non-Markovian problem they cannot, in general, be reconstructed from the reduced trajectory alone because the first operator insertion changes the subsequent joint system–environment evolution. PT-MPO calculations of such correlations have been developed for strongly coupled spin chains and structured baths [39], and have since been used in process-tensor approaches to nonlinear spectroscopy [40]. We take a minimal two-spin analogue so that every result can also easily be checked against direct joint evolution,

$$H_S = S_x, \qquad H_E = S_x, \qquad H_{SE} = S_z \otimes S_z, \tag{84}$$

with the system initially in $|\downarrow\rangle$ and the environment in $|\uparrow\rangle$.

For $t_2 \geq t_1$, the ordered correlation is

$$C_{AB}(t_2, t_1) = \mathrm{Tr}_{SE}\big[(A \otimes \mathbb{I}_E)U_{2:1}(B \otimes \mathbb{I}_E)U_{1:0}\rho_{SE}(0)U_{1:0}^\dagger U_{2:1}^\dagger\big]. \tag{85}$$

Thus B is inserted from the left at t_1, the resulting operator is propagated to t_2, and A closes the final leg. The order is reversed when $t_1 > t_2$, while coincident insertions are combined on the same temporal leg. These cases are encoded by `two_time_correlation_seq`, so a complete grid reuses one process tensor:

Julia

```julia
Czz = [
    evaluate_process(
        process_tensor,
        two_time_correlation_seq(
            process_tensor, (sigma_z, n2), (sigma_z, n1);
            rho0=rho_S0,
        ),
    )
    for n1 in time_indices, n2 in time_indices
]
```

Listing 20: Two-time correlation grid from one stored process tensor.

For $t_2 > t_1$, the helper translates Eq. (85) into instruments: $\rho_S(0)$ is prepared at the first input, B is inserted as a left action at t_1, A is inserted as a left action at t_2, and a terminal `trace_out` realises the partial trace. On a five-step process with $(n_2, n_1) = (3, 1)$, the resulting schedule is

Julia

```julia
seq = two_time_correlation_seq(
    process_tensor, (sigma_z, 3), (sigma_z, 1);
    rho0=rho_S0,
)
println(seq)
```

Terminal

```
InstrumentSeq(default=IdentityOperation, nsteps=5, 4 explicit entries)
  tstep=0 => StatePreparation{MPO{Hilbert, Nothing}}
  tstep=2 => LeftRightOperator{MPO{Hilbert, Nothing}, MPO{Hilbert,
  ↪  Nothing}}
  tstep=4 => LeftRightOperator{MPO{Hilbert, Nothing}, MPO{Hilbert,
  ↪  Nothing}}
  tstep=5 => TraceOut
```

Listing 21: Instrument sequence for $C_{zz}(t_3, t_1)$ on a five-step process.

The default identity fills the idle slots 1 and 3. The left-action at `tstep=2` is $B = \sigma_z$ at t_1, and that at `tstep=4` is $A = \sigma_z$ at t_2. If $t_1 > t_2$, the early insertion becomes a right action; if $t_1 = t_2$, the two operators are multiplied on the same temporal leg.

Several expectations follow without solving the dynamics. Hermiticity and $\sigma_z^2 = \mathbb{I}$ give

$$C_{zz}(t, t) = 1, \qquad C_{zz}(t_2, t_1)^* = C_{zz}(t_1, t_2), \tag{86}$$

so the real and imaginary parts are respectively symmetric and antisymmetric under $t_1 \leftrightarrow t_2$. The equal-time cross-correlations instead follow from

$$\sigma_z \sigma_x = i\sigma_y, \qquad \sigma_x \sigma_y = i\sigma_z. \tag{87}$$

So their real part of the same-time correlation should be zero. The numerical grids in Fig. 12 reproduce these correlation properties faithfully.

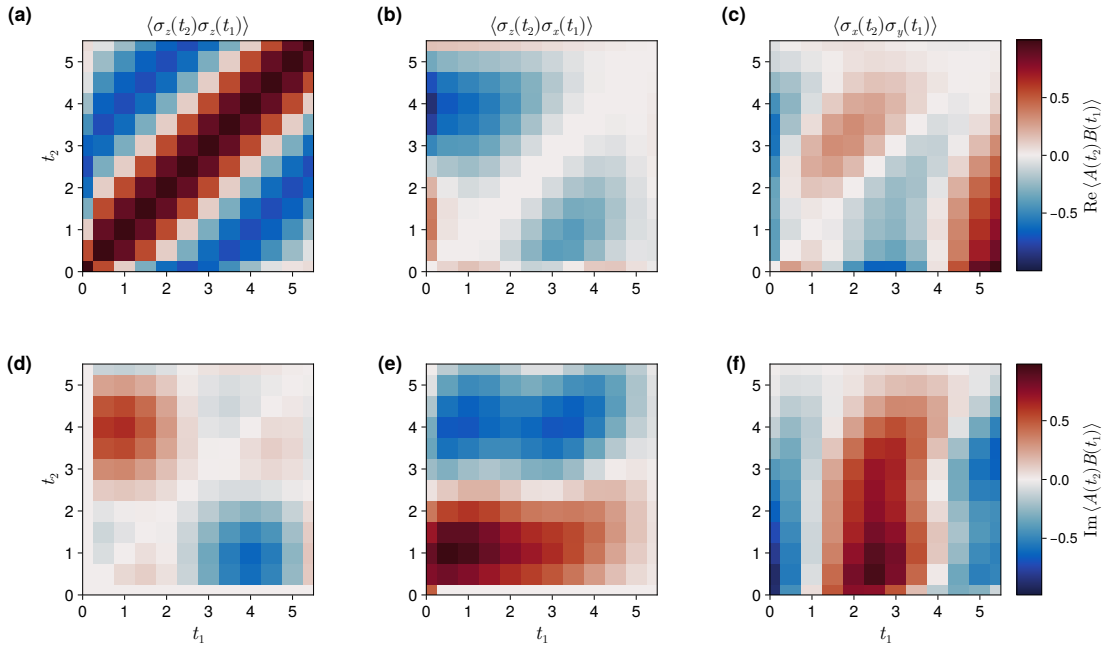

Figure 12: **Two-time spin correlations from a reused process tensor.** Panels (a)–(c) show the real parts of C_{zz}, C_{zx}, and C_{xy} for the two-spin model in Eq. (84); (d)–(f) show the corresponding imaginary parts. The autocorrelation (a) has a unit real diagonal and the conjugate symmetry of Eq. (86) in (d); the cross-correlation diagonals follow Eq. (87). Every pixel is obtained by changing only the two operator-insertion times of the same stored process tensor.

5.4 Memory-assisted control with a quantum tester

The previous experiments repeatedly removed the system's local memory. We finally consider the opposite setting, in which the intervention deliberately retains a quantum memory. Quantum testers and combs were introduced to describe the most general probing strategies for channels used over several times, including protocols in which an ancilla persists between uses [30, 41]. Such memory-assisted strategies are particularly natural for correlated circuit noise, whose implications for randomized benchmarking have been analysed in Refs. [34, 35].

We return to the thermal spin–boson process of Eq. (79), but attach an isolated ancillary qubit A to the noisy processor qubit S. The protocol is as follows: first SWAP the system and ancilla to store the logical state in A, then perform Z_A operation only on the ancilla while it is parked away from the bath, and then do the second SWAP to retrieve it. The persistent ancillary wire turns this sequence into a temporally correlated tester.

Julia

```
memory = tester(ancilla_sites, rho_A0)

U_SA = op("SWAP", only(system_sites), only(ancilla_sites))
U_A = op("Z", only(ancilla_sites))
joint_U_SA = joint_unitary(U_SA, system_sites, ancilla_sites)
local_U_A = tester_unitary(U_A, ancilla_sites)

controls = TesterSeq(nsteps=process_tensor.nsteps)
controls += joint_U_SA, store_step
```

```
controls += local_U_A, phase_step
controls += joint_U_SA, retrieve_step

trajectory = evolve(
    process_tensor, rho_S0;
    tester=memory, tester_seq=controls, return_joint=true,
)
```

Listing 22: SWAP–phase–SWAP tester sequence for memory-assisted control.

$$I(S{:}A) = S(\rho_S) + S(\rho_A) - S(\rho_{SA}), \qquad S(\rho) = -\mathrm{Tr}\big(\rho \log_2 \rho\big). \tag{88}$$

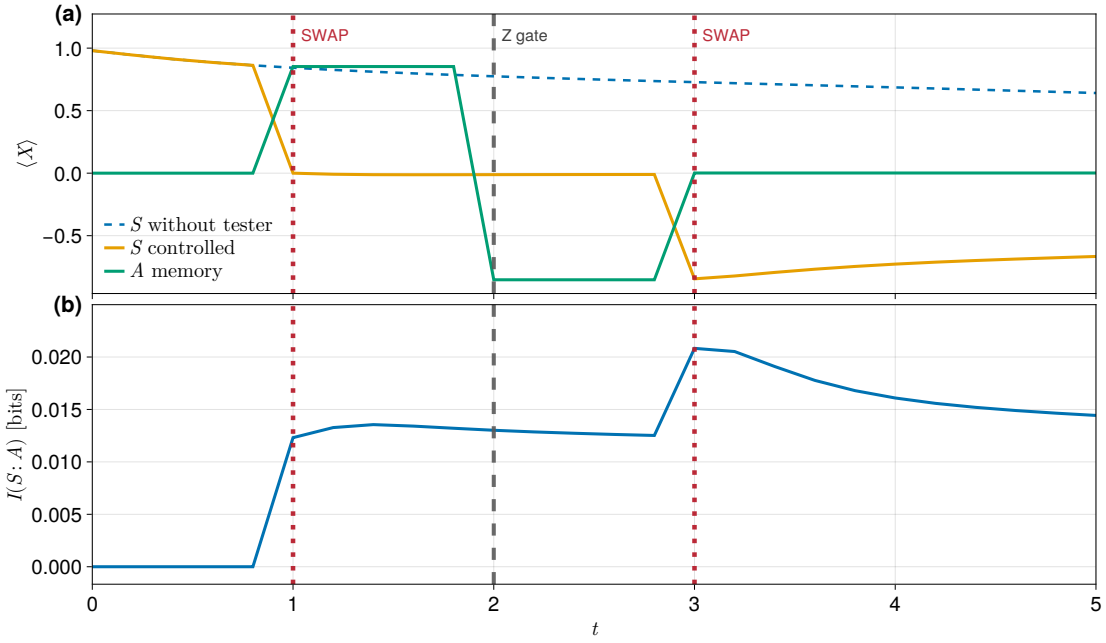

Figure 13: **(a) Transverse magnetisation of the SWAP–phase–SWAP protocol.** The first SWAP transfers $\langle X \rangle$ from the noisy processor S to the isolated ancilla A, the Z_A gate reverses its sign, and the second SWAP returns it; the uncontrolled system continues to dephase on the processor. **(b) System–ancilla correlation.** The joint operations generate a nonzero mutual information $I(S{:}A)$, showing that the two reduced trajectories are not an independent product description of the protocol.

The output of evolving a PT-MPO with a tester instrument contains the reduced states $\rho_S(t)$ and $\rho_A(t)$ by default. We can also obtain the joint trajectory $\rho_{SA}(t)$ by setting the `return_joint` keyword argument to `true`. We use the joint system-ancillary state to quantify the total correlations between the system and ancilla by the quantum mutual information

$$I(S{:}A) = S(\rho_S) + S(\rho_A) - S(\rho_{SA}), \qquad S(\rho) = -\mathrm{Tr}\big(\rho \log_2 \rho\big). \tag{89}$$

The panels of Fig. 13 illustrate the process using our defined protocol. If we leave the system to interact with the environment without any intervention, the system will dephase. However, introducing an ancillary qubit and performing joint operations on the system and ancilla shows how the qubit information moves between these two wires, while generating correlations between them as a consequence of the noise the system experiences. The distinction between the

two memories is important: environmental memory is already compressed into the PT-MPO, whereas the controllable memory of the experimenter is carried explicitly by the tester ancilla.

Taken together, these workflows move from an unobserved reduced trajectory to outcome-resolved causal breaks, operator insertions at two times, and finally a coherent intervention with its own memory. Their common implementation is the central design choice of `ProcessTensors.jl`: the expensive environment is encoded once, while the experiment is changed by replacing the instruments attached to its temporal legs.

6 Benchmarks of the ACE algorithm

7 Diagnosing memory in process tensors

7.1 Temporal cuts and memory spaces

7.2 Memory spectrum and temporal compressibility

7.3 Memory diagnosis in a non-Markovian example

8 Extensibility of `ProcessTensors.jl` and its roadmap

8.1 A common interface for process-tensor builders

8.2 Gaussian influence-functional builders

8.3 Chain-mapped and explicit-environment constructions

8.4 Fermionic process tensors

8.5 Characterising and reconstructing process tensors

8.6 Quantum memory witness generation

9 Conclusion

Acknowledgements

Acknowledgements should follow immediately after the conclusion.

Disclosure of generative AI use The authors disclose that the following generative AI tools have been used in the preparation of this submission. Large language models accessed through coding assistants (including Cursor and ChatGPT) were used to draft `TikZ` figures, to generate example code listings that were subsequently verified by hand, and to assist in the construction of `ProcessTensors.jl` package. All AI-generated source, including figure code, listing code, and package code, was examined thoroughly by the authors, tested against the intended mathematical and numerical behaviour, and is not left in an unverified "vibe-coded" state. The authors take full responsibility for the contents of the manuscript and of the software.

Author contributions This is optional. If desired, contributions should be succinctly described in a single short paragraph, using author initials.

Funding information Authors are required to provide funding information, including relevant agencies and grant numbers with linked author's initials. Correctly-provided data will be linked to funders listed in the Fundref registry.

Code and data availability The development repository is available at https://github.com/Gauthameshwar/ProcessTensors.jl and the documentation at https://gauthameshwar.github.io/ProcessTensors.jl/stable/. The final paper will cite an immutable release and archive the scripts/data used for all benchmark figures. **[TODO: insert release DOI/Zenodo or SciPost codebase DOI at submission.]**

References

[1] V. Gorini, A. Kossakowski and E. C. G. Sudarshan, *Completely positive dynamical semigroups of N-level systems*, Journal of Mathematical Physics **17**(5), 821 (1976), doi:10.1063/1.522979.

[2] G. Lindblad, *On the generators of quantum dynamical semigroups*, Communications in Mathematical Physics **48**(2), 119 (1976), doi:10.1007/BF01608499.

[3] Á. Rivas, S. F. Huelga and M. B. Plenio, *Quantum non-markovianity: Characterization, quantification and detection*, Reports on Progress in Physics **77**(9), 094001 (2014), doi:10.1088/0034-4885/77/9/094001.

[4] I. de Vega and D. Alonso, *Dynamics of non-markovian open quantum systems*, Reviews of Modern Physics **89**(1), 015001 (2017), doi:10.1103/RevModPhys.89.015001.

[5] F. A. Pollock, C. Rodríguez-Rosario, T. Frauenheim, M. Paternostro and K. Modi, *Operational markov condition for quantum processes*, Physical Review Letters **120**(4), 040405 (2018), doi:10.1103/PhysRevLett.120.040405.

[6] F. A. Pollock, C. Rodríguez-Rosario, T. Frauenheim, M. Paternostro and K. Modi, *Non-Markovian quantum processes: Complete framework and efficient characterisation*, Physical Review A **97**(1), 012127 (2018), doi:10.1103/PhysRevA.97.012127.

[7] S. Milz and K. Modi, *Quantum stochastic processes and quantum non-Markovian phenomena*, PRX Quantum **2**(3), 030201 (2021), doi:10.1103/PRXQuantum.2.030201.

[8] P. Taranto, S. Milz, M. Murao, M. T. Quintino and K. Modi, *Higher-order quantum operations*, doi:10.48550/arXiv.2503.09693 (2025), 2503.09693.

[9] M. R. Jørgensen and F. A. Pollock, *Exploiting the causal tensor network structure of quantum processes to efficiently simulate non-markovian path integrals*, Physical Review Letters **123**(24), 240602 (2019), doi:10.1103/PhysRevLett.123.240602.

[10] A. Strathearn, P. Kirton, D. Kilda, J. Keeling and B. W. Lovett, *Efficient non-markovian quantum dynamics using time-evolving matrix product operators*, Nature Communications **9**, 3322 (2018), doi:10.1038/s41467-018-05617-3.

[11] M. Cygorek, M. Cosacchi, A. Vagov, V. M. Axt, B. W. Lovett, J. Keeling and E. M. Gauger, *Simulation of open quantum systems by automated compression of arbitrary environments*, Nature Physics **18**, 662 (2022), doi:10.1038/s41567-022-01544-9.

[12] V. Link, H.-H. Tu and W. T. Strunz, *Open quantum system dynamics from infinite tensor network contraction*, Physical Review Letters **132**(20), 200403 (2024), doi:10.1103/PhysRevLett.132.200403, 2307.01802.

[13] G. E. Fux, P. Fowler-Wright, J. Beckles, E. P. Butler, P. R. Eastham, D. Gribben, J. Keeling, D. Kilda, P. Kirton, E. D. C. Lawrence, B. W. Lovett, E. O'Neill *et al.*, *OQuPy: A Python package to efficiently simulate non-markovian open quantum systems with process tensors*, The Journal of Chemical Physics **161**(12), 124108 (2024), doi:10.1063/5.0225367.

[14] M. Cygorek and E. M. Gauger, *ACE: A general-purpose non-markovian open quantum systems simulation toolkit based on process tensors*, The Journal of Chemical Physics **161**(7), 074111 (2024), doi:10.1063/5.0221182.

[15] A. Bose, *QuantumDynamics.jl: A modular approach to simulations of dynamics of open quantum systems*, The Journal of Chemical Physics **158**(20), 204113 (2023), doi:10.1063/5.0151483.

[16] T. Lacroix, B. Le Dé, A. Riva, A. J. Dunnett and A. W. Chin, *MPSDynamics.jl: Tensor network simulations for finite-temperature (non-markovian) open quantum system dynamics*, The Journal of Chemical Physics **161**(8), 084116 (2024), doi:10.1063/5.0223107.

[17] J. Keeling, E. M. Stoudenmire, M.-C. Bañuls and D. R. Reichman, *Process tensor approaches to non-markovian quantum dynamics*, Physical Review X **16**, 020502 (2026), doi:10.1103/1ncg-11hz.

[18] T. Lacroix, A. Burgess, N. Lorenzoni, J. Wiercinski, K. Damezin, J. Lim, D. Tamascelli, A. W. Chin, M. Cygorek, B. W. Lovett, J. Keeling, S. F. Huelga *et al.*, *Tensor network methods for non-perturbative dynamics of open quantum systems*, doi:10.48550/arXiv.2608.09850 (2026), 2608.09850.

[19] C. J. Wood, J. D. Biamonte and D. G. Cory, *Tensor networks and graphical calculus for open quantum systems*, Quantum Information and Computation **15**(9-10), 759 (2015), doi:10.26421/QIC15.9-10-3, 1111.6950.

[20] M.-D. Choi, *Completely positive linear maps on complex matrices*, Linear Algebra and its Applications **10**(3), 285 (1975), doi:10.1016/0024-3795(75)90075-0.

[21] A. Jamiołkowski, *Linear transformations which preserve trace and positive semidefiniteness of operators*, Reports on Mathematical Physics **3**(4), 275 (1972), doi:10.1016/0034-4877(72)90011-0.

[22] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 10th anniversary edn., ISBN 9781107002173, doi:10.1017/CBO9780511976667 (2010).

[23] J. Watrous, *The Theory of Quantum Information*, Cambridge University Press, doi:10.1017/9781316848142 (2018).

[24] U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, Annals of Physics **326**(1), 96 (2011), doi:10.1016/j.aop.2010.09.012.

[25] R. Orús, *A practical introduction to tensor networks: Matrix product states and projected entangled pair states*, Annals of Physics **349**, 117 (2014), doi:10.1016/j.aop.2014.06.013.

[26] M. Fishman, S. R. White and E. M. Stoudenmire, *The ITensor software library for tensor network calculations*, SciPost Physics Codebases p. 4 (2022), doi:10.21468/SciPostPhysCodeb.4.

[27] M. Cygorek, B. W. Lovett, J. Keeling and E. M. Gauger, *Treelike process tensor contraction for automated compression of environments*, Physical Review Research **6**(4), 043203 (2024), doi:10.1103/PhysRevResearch.6.043203.

[28] E. B. Davies and J. T. Lewis, *An operational approach to quantum probability*, Communications in Mathematical Physics **17**(3), 239 (1970), doi:10.1007/BF01647093.

[29] M. Ozawa, *Quantum measuring processes of continuous observables*, Journal of Mathematical Physics **25**(1), 79 (1984), doi:10.1063/1.526000.

[30] G. Chiribella, G. M. D'Ariano and P. Perinotti, *Theoretical framework for quantum networks*, Physical Review A **80**(2), 022339 (2009), doi:10.1103/PhysRevA.80.022339.

[31] G. A. L. White, P. Jurcevic, C. D. Hill and K. Modi, *Unifying non-Markovian characterization with an efficient and self-consistent framework*, Physical Review X **15**(2), 021047 (2025), doi:10.1103/PhysRevX.15.021047, 2312.08454.

[32] M. Ziman, *Process positive-operator-valued measure: A mathematical framework for the description of process tomography experiments*, Physical Review A **77**(6), 062112 (2008), doi:10.1103/PhysRevA.77.062112, 0802.3862.

[33] N. F. Ramsey, *A molecular beam resonance method with separated oscillating fields*, Physical Review **78**(6), 695 (1950), doi:10.1103/PhysRev.78.695.

[34] P. Figueroa-Romero, K. Modi, R. J. Harris, T. M. Stace and M.-H. Hsieh, *Randomized benchmarking for non-Markovian noise*, PRX Quantum **2**(4), 040351 (2021), doi:10.1103/PRXQuantum.2.040351.

[35] S. Gandhari and M. J. Gullans, *Quantum non-Markovian noise in randomized benchmarking of spin-boson models*, Physical Review Research **8**(2), 023075 (2026), doi:10.1103/z8zh-n1hl, 2502.14702.

[36] A. J. Leggett, S. Chakravarty, A. T. Dorsey, M. P. A. Fisher, A. Garg and W. Zwerger, *Dynamics of the dissipative two-state system*, Reviews of Modern Physics **59**(1), 1 (1987), doi:10.1103/RevModPhys.59.1.

[37] G. A. L. White, C. D. Hill, F. A. Pollock, L. C. L. Hollenberg and K. Modi, *Demonstration of non-markovian process characterisation and control on a quantum processor*, Nature Communications **11**(1), 6301 (2020), doi:10.1038/s41467-020-20113-3.

[38] G. A. L. White, F. A. Pollock, L. C. L. Hollenberg, K. Modi and C. D. Hill, *Non-Markovian quantum process tomography*, PRX Quantum **3**(2), 020344 (2022), doi:10.1103/PRXQuantum.3.020344.

[39] G. E. Fux, D. Kilda, B. W. Lovett and J. Keeling, *Tensor network simulation of chains of non-Markovian open quantum systems*, Physical Review Research **5**(3), 033078 (2023), doi:10.1103/PhysRevResearch.5.033078, 2201.05529.

[40] R. de Wit, J. Keeling, B. W. Lovett and A. W. Chin, *Process tensor approaches to modeling two-dimensional spectroscopy*, Physical Review Research **7**(1), 013209 (2025), doi:10.1103/PhysRevResearch.7.013209, 2402.15454.

[41] G. Chiribella, G. M. D'Ariano and P. Perinotti, *Memory effects in quantum channel discrimination*, Physical Review Letters **101**(18), 180501 (2008), doi:10.1103/PhysRevLett.101.180501, 0803.3237.

[42] S. Paeckel, T. Köhler, A. Swoboda, S. R. Manmana, U. Schollwöck and C. Hubig, *Time-evolution methods for matrix-product states*, Annals of Physics **411**, 167998 (2019), doi:10.1016/j.aop.2019.167998.

A How to read the Julia listings

Code examples throughout this paper are displayed in framed Julia listings. A blue **Julia** badge in the top-right corner identifies the language. Printed program output is shown in a matching gray box with a **Terminal** badge; that text is not syntax-highlighted. Line numbers on the left are the ones referred to in the text (for example, Listing 23, lines 4–6). Syntax highlighting is chosen to make the origin of each identifier visible at a glance, rather than to reproduce a particular editor theme.

The colour roles are as follows.

- **Blue, bold:** Julia keywords such as `using`.
- **Violet, bold:** names exported by `ProcessTensors.jl`. This includes package-specific constructors and converters (`to_dm`, `liouv_sites`, `liouvillian_mpo`, `tester`, ...), tensor-network types such as `MPS`, `MPO`, and `OpSum`, and re-exported ITensorMPS inspection routines such as `siteinds`, `linkinds`, and `inner`.
- **Teal, bold:** functions that belong to `ITensors.jl` and are *not* part of the `ProcessTensors.jl` export list (`Index`, `ITensor`, `dag`, `prime`, `delta`, `op`, `apply`, ...).
- **Orange, bold:** package types and backends such as `Hilbert`, `Liouville`, `Dense`, `ACE`, `InstrumentSeq`, and `TesterSeq`.
- **Near-black:** ordinary variables.
- **Green, bold:** numeric literals.
- **Crimson:** string literals.
- **Grey italic:** comments.

Listing 23 collects these conventions in one place. Violet names are the objects a user of `ProcessTensors.jl` is expected to call; teal names are the underlying ITensor primitives on which those objects are built.

Julia

```
using ITensors, ProcessTensors

s = Index(2, "s")                          # ITensor index
psi = ITensor([1.0, 0.0], s)
sites = siteinds("S=1/2", 1)

rho = to_dm(MPS(sites, ["Up"]))            # ProcessTensors converter +
 ↪  MPS
H = OpSum()
```

```
H += 0.7, "Sx", 1
L = liouvillian_mpo(H, liouv_sites(sites))
```

Listing 23: Colour legend for the listings in this paper.

B Time dynamics in Liouville space

The Liouville-space MPS and MPO objects introduced in Sec. 2.4 provide a direct representation of time-dependent open-system states and generators. If $|\rho(0)\rangle\rangle$ is an initial Liouville MPS and $\mathcal{L}$ is its Liouvillian, the state at time t is

$$|\rho(t)\rangle\rangle = \exp(t\mathcal{L})|\rho(0)\rangle\rangle. \tag{B.1}$$

`ProcessTensors.jl` extends the time-evolution algorithms provided by `ITensorMPS.jl` to its typed Hilbert- and Liouville-space objects. These routines are useful for time-local master equations and provide reference trajectories against which process-tensor calculations can be validated.

For a small two-spin example, Liouville-space TEBD approximates the propagator over successive time steps by constructing local gates for $\exp(\Delta t\,\mathcal{L})$ from a Hilbert-space Hamiltonian and local dissipators:

Julia

```julia
using ITensors, ProcessTensors
using ITensors.Ops: Trotter

sites = siteinds("S=1/2", 2)
sites_L = liouv_sites(sites)
psi0 = MPS(sites, ["Up", "Dn"])
rho_L0 = to_liouville(to_dm(psi0); sites=sites_L)

H = OpSum()
H += 1.0, "Sz", 1, "Sz", 2
H += 0.4, "Sx", 1
H += 0.4, "Sx", 2
jumps = [(0.15, "S-", 1), (0.15, "S-", 2)]

dt, T = 0.025, 0.1
rho_L_tebd = tebd(
    rho_L0, H, dt, T;
    jump_ops=jumps,
    alg=Trotter{2}(),
    maxdim=32,
    cutoff=1e-12,
)
```

Listing 24: Time evolution of a dissipative Liouville-space state with TEBD.

TDVP provides a complementary time-evolution route by acting with the Liouvillian MPO itself. The evolution time supplied to TDVP is real because the Hamiltonian contribution $-i[H,\rho]$ and all dissipative terms are already contained in $\mathcal{L}$:

Julia

```julia
L = liouvillian_mpo(H, sites_L; jump_ops=jumps)
rho_L_tdvp = tdvp(
    L, T, rho_L0;
    time_step=dt,
    nsite=2,
    maxdim=32,
    cutoff=1e-12,
    outputlevel=0,
)
```

Listing 25: Time evolution of the same Liouville-space state with TDVP.

The two methods expose the same physical time dynamics through different tensor-network approximations: TEBD controls the local Trotter decomposition, whereas TDVP controls projection onto the evolving MPS manifold. In either case, convergence should be checked with respect to the time step, truncation cutoff, and maximum bond dimension. The resulting trajectory should also be validated by monitoring trace, Hermiticity, and positivity.

Complete discussions of Trotter order, TDVP parameters, dense validation, and time-resolved observables are available in the unitary-dynamics and dissipative-dynamics tutorials. Larger time-dependent examples, including dissipative spin chains, boundary-driven transport, and driven bosonic systems, are linked from the stable package documentation. For a more detailed overview of time evolution using MPS and MPO, refer to the review by Paeckel et al. [42].